

Decoding Employer Preferences: A Natural Language
Processing Analysis of Job Descriptions

Junyi Xu, Ruolan Ren, Angel Hu, Xuejing Yan

May 2, 2026

Abstract

This project examines what traits firms value most when evaluating job candidates. We used the “Scraped Job Descriptions” dataset, and we analyzed the job descriptions collected from an online job listing platform. The dataset contains textual descriptions of job responsibilities, qualifications, and requirements across multiple occupational categories. To make the analysis more systematic, we divided the job postings into four major categories: business and administration, information and communication, science and engineering, and administrative and commercial.

We applied several natural language processing methods from Module 2, 3, 4, 5, 7, 8, 9, 11 to identify patterns in employer preferences. First, we cleaned and preprocessed the text by lowercasing, removing punctuation, tokenizing, removing stopwords, and lemmatizing words. We then used word frequency analysis, bigram analysis, TF-IDF, part-of-speech analysis, topic modeling, and trait-based lexicon analysis to examine both technical qualifications and personal traits mentioned in job descriptions.

1.Introduction

When employers write job descriptions, they consider not only the responsibilities of a position but also the qualifications, skills, and personal traits they expect from candidates. These descriptions therefore provide useful evidence about what firms value in the labor market. While job descriptions often include technical requirements such as degrees, certifications, and years of experience, they also frequently mention soft skills such as communication, teamwork, leadership, adaptability, and problem-solving. So our project uses natural language processing methods to study employer preferences in job descriptions. Our research question is:

What traits do firms value most when evaluating candidates?

This question is important because job descriptions shape how applicants present themselves and how firms define an ideal candidate. By analyzing the language used in job advertisements, we can better understand whether firms prioritize technical knowledge, interpersonal ability, managerial capacity, or problem-solving skills. This is especially useful because different occupational categories may value different traits. For example, technology-related jobs may emphasize problem-solving and technical expertise, while business or administrative roles may place more emphasis on management, communication, and client-facing skills.

To answer this question, we analyze the “Scraped Job Descriptions” dataset from an online job listing platform. We divide the data into four occupational categories: business and administration, information and communication, science and engineering, and administrative and commercial. Each group member analyzes one category using NLP methods.

This project contributes to understanding how firms describe ideal candidates across different professional fields. Rather than relying on individual examples of job postings, we use text analysis to identify broader patterns across hundreds or thousands of job descriptions. The results show that firms value both job-specific technical skills and general workplace traits, but the relative importance of these traits varies across categories.

2.Data Description

The dataset used in this project is the "Scraped Job Descriptions" dataset available on Kaggle(<https://www.kaggle.com/datasets/marcocavaco/scraped-job-descriptions>). This dataset contains approximately 4,413 job descriptions, which were obtained by web scraping from an online recruitment platform. Each record represents a job posting and includes text information such as job responsibilities, requirements, preferred skills, and the company's expectations for the candidate.

The main variable we used in our analysis was the job description text. This type of text is highly suitable for natural language processing as it contains the specific requirements of the employer for the candidate. The job description includes both formal conditions, such as educational background, work experience, technical skills and industry knowledge, as well as personal traits, such as communication skills, teamwork, leadership, problem-solving ability, adaptability and attention to detail.

We divided the four occupational categories separately in order to compare the differences in employer preferences among different types of positions. This division is crucial because different occupations may emphasize different candidate traits. Before conducting the formal analysis, we also preprocessed the text, including converting to lowercase, removing punctuation and irrelevant symbols, tokenizing, removing stop words, and restoring the words to their basic forms.

3. Methodology

Business and Administration

For the "Business and Administration" section, we analyzed the job descriptions classified as "Business and Administration Professionals" and "Business and Administration Associate Professionals". We chose these two categories because they are directly relevant to our research question: What traits do companies value most when evaluating candidates for business and administrative positions? After filtering the data set, we created a more focused sub-data set that can be analyzed separately from other job categories.

Firstly, we carried out preprocessing on the text. Since the original job descriptions usually contain inconsistent formats and some unnecessary symbols, we converted all the job descriptions to lowercase, removed punctuation marks, extra spaces, numbers, and non-alphabetic characters, and split the text into individual words. We also deleted common stop words such as "the," "and," and "for," and used word normalization, so that related word forms like "managing," "managed," and "management" could be processed more consistently.

After completing the preprocessing, we employed several NLP methods. Firstly, we conducted word frequency analysis and bigram analysis to identify the most common words and recurring two-word phrases in the job descriptions of business and administrative positions. These methods helped us discover patterns related to communication skills, teamwork, management abilities, and experience requirements. Then, we used TF-IDF to compare this subcategory with other job categories, thereby identifying which words are more representative in business and administrative positions. Finally, we used the LDA topic model to discover larger themes within this sub-data set, such as auditing/accounting, sales/marketing, bookkeeping, and administrative support.

Overall, this method combines text preprocessing, descriptive text analysis, comparative weight analysis, and unsupervised topic modeling. By using multiple NLP techniques, we can support the research findings from various perspectives: including common words, recurring phrases, distinguishing keywords, and broader topic clustering.

Information and Communication

For the "Information and Communication" section, we selected job descriptions labeled as "Information and Communications Technology Professionals" or "Information and Communications Technicians", resulting in a total of 541 job descriptions. This sub-data set was

used to analyze the language employers used when describing ICT-related positions, as well as the traits they emphasized in selecting candidates.

First, we cleaned the text, which included converting all job descriptions to lowercase, removing punctuation marks and unnecessary symbols, segmenting the text, deleting stop words, and restoring the words to their basic forms. This made the job description text more uniform and prepared it for the subsequent NLP analysis.

Next, we employed four NLP methods. Firstly, word frequency analysis was used to identify the most common words in the ICT job descriptions; bigram analysis was used to capture common two-word phrases, as these phrases provide more context than individual words. Then, we used TF-IDF analysis to identify the particularly important words in the Information and Communication category. Finally, we used trait lexicon analysis to measure the frequency of different candidate traits appearing in the job descriptions.

In the trait lexicon analysis, we created a candidate trait dictionary, including communication, teamwork, problem-solving, detail orientation, leadership, adaptability, creativity, independence and time management. Each trait is represented by a group of related keywords. If a job description contains one or more keywords related to a certain trait, we count it as mentioning that trait. Afterwards, we calculate the number and percentage of job descriptions in which each trait is mentioned, thereby comparing which candidate qualities are most emphasized by the company in ICT-related positions.

Science and Engineering Professionals

In the "Science and Engineering Professionals" section, we mainly analyzed whether there were significant language differences in the job descriptions of science and engineering positions. Similar to before, we first cleaned the text, including converting to lowercase, removing URLs, email addresses, non-alphabetic characters, stop words, and short words, in order to reduce text noise.

After cleaning, we used CountVectorizer to extract unigrams and bigrams to observe common words and two-word phrases. We also used TF-IDF to identify more representative words in different job titles, as it can better highlight the key words in specific positions than ordinary word frequency.

Next, we conducted supervised text classification to test whether job labels could be predicted based solely on the job descriptions. We compared the CountVectorizer + Multinomial Naive Bayes and TF-IDF + Logistic Regression models and used a stratified 80/20 train-test split. At

the same time, we also trained a Naive Bayes model across four main occupational categories to determine if there are language differences between the broad categories.

To make the results more stable, we further used 5-fold stratified cross-validation. The main metric was the macro-averaged F1 score, as it is more suitable for imbalanced data, while accuracy was used as a supplementary reference.

Finally, we used LDA topic modeling for unsupervised analysis. LDA identifies potential topics based on word co-occurrence. We set the number of topics to four and compared the dominant topic of each job description with major categories and specific job titles.

Administration and Commercial

In the "Administration and Commercial" section, we mainly analyzed the job descriptions under the "Administrative and Commercial Managers" category, resulting in a total of 321 job descriptions. This category encompasses positions related to finance, sales, marketing, operations, management, and administrative leadership, making it suitable for analyzing the candidate traits that employers value in management and commercial positions.

Just like in the previous sections, we first carried out preprocessing on the text, including converting to lowercase, removing irrelevant symbols, tokenization, deleting stop words, and performing word normalization. This step was mainly to make the recruitment text more uniform, facilitating subsequent NLP analysis.

Subsequently, we conducted a job title frequency analysis to understand which positions appeared most frequently in this sample and the composition of positions within this category. Then, we employed trait lexicon analysis by calculating the frequency of keywords related to the candidates' abilities and traits, in order to directly address our research questions.

To further analyze the language patterns, we also employed bigram analysis, POS tagging, TF-IDF, and LDA topic modeling. Bigram analysis identified common phrases such as "customer service," "digital marketing," and "attention to detail." POS tagging mainly focused on the adjectives that appeared in the job requirements and the descriptions of ideal candidates. TF-IDF was used to identify more representative keywords, while LDA revealed broader themes within the category, such as financial control, business management, sales and marketing, and operational services.

4. Analysis and Results:

As mentioned earlier, we divided the job descriptions into four occupational categories: Business and Administration, Information and Communication, Science and Engineering Professionals, and Administration and Commercial. Since these categories represent different types of work, we did not apply the exact same NLP methods to every group. Instead, we selected the methods that best fit the structure, content, and research focus of each category.

Business and Administration

In the Business and Administration category, we find that employers care a lot about communication skills, coordination, management ability, and practical business experience. From the frequency analysis, we realize that many job descriptions in this category are centered on daily business operations and workplace responsibilities. Common words include “communication,” “management,” “support,” “service,” “team,” “client,” “customer,” “experience,” and “organization.” We think this shows that companies are not only looking for candidates who can complete administrative tasks, but also people who can work well with others, communicate clearly, and help the organization run smoothly.

Looking at the repeated two-word phrases gives us more context than single-word counts alone. Many common phrases are related to business operations, customer service, project management, administrative support, and communication skills. From this, we find that business and administration roles often sit between internal coordination and external service. Candidates are expected to manage information, interact with customers or clients, organize work processes, and support management or operational goals. So we do not think these jobs should be understood as only paperwork or routine office tasks. They also require interpersonal judgment, organizational awareness, and the ability to coordinate across teams.

After comparing this category with other job groups through TF-IDF, we can see which words are more representative of Business and Administration. These terms are mainly related to accounting, auditing, sales, bookkeeping, administration, client service, and business support. This makes us realize that Business and Administration is not one completely unified field. Instead, it includes several related but different job directions. Some roles focus more on finance and accounting, while others focus more on sales, customer relations, or administrative operations.

The topic model adds another layer to this finding. It identifies several major themes within the category, including audit and accounting, sales and marketing, bookkeeping, and administrative support. These themes suggest that there is a clear internal structure within the category. Some positions focus on numerical accuracy, financial records, and compliance-related tasks. Others emphasize marketing, sales, and customer-facing communication. Another group focuses more

on administrative coordination and office support. This also explains why both technical business terms and interpersonal trait-related words appear in the results.

Overall, we find that employers in the Business and Administration category value a combination of soft skills and practical business skills. Words related to communication, teamwork, organization, and management appear frequently, which suggests that companies want candidates who can collaborate and coordinate effectively. At the same time, words such as “accounting,” “audit,” “sales,” and “administrative” show that employers also expect candidates to have job-specific knowledge. The main conclusion is that candidates in business and administration are evaluated not only by their technical qualifications, but also by whether they can function well in an organizational environment.

Information and Communication

In the Information and Communication category, we find that employers place strong emphasis on technical knowledge, problem-solving, system-related work, and the ability to explain technical information. Compared with Business and Administration, this category is more specialized and technical. However, we also notice that communication, teamwork, and attention to detail still appear as important workplace traits.

From the most frequent words, we can see that many descriptions are centered on software, systems, data, support, development, technology, technical work, projects, users, and services. We think this reflects the daily responsibilities of many ICT roles, which often involve digital systems, software tools, technical infrastructure, or user support. The appearance of words such as “support” and “service” is also meaningful because it suggests that ICT work is not always isolated technical work. Many positions require employees to help users, explain technical problems, maintain systems, and collaborate with other departments.

The repeated two-word phrases give a clearer picture of common tasks and requirements in ICT roles. Common phrases include “software development,” “technical support,” “information systems,” “project management,” “data analysis,” “computer systems,” and “problem solving.” These phrases show that employers expect candidates to perform technical work, but also to have analytical and coordination skills. For example, “software development” and “information systems” reflect technical expertise, while “project management” and “technical support” suggest that ICT employees also need communication and organizational skills.

When we use TF-IDF to compare this category with others, the more representative words are mostly related to software, systems, and data. We think this makes sense because ICT job descriptions usually need to specify the technical tools, systems, or skills required for the position. At the same time, technical vocabulary does not replace soft-skill language. Instead,

technical terms and workplace trait terms often appear together. This suggests that companies expect ICT candidates to have technical ability, but also to work effectively with other people.

The trait lexicon analysis makes this point even clearer. Among the traits we analyzed, problem-solving, communication, teamwork, detail orientation, adaptability, independence, creativity, leadership, and time management all appear in ICT job descriptions. Problem-solving stands out as especially important because many ICT jobs involve diagnosing errors, improving systems, solving user problems, or creating technical solutions. Communication is also important because technical workers often need to explain complex information to non-technical users, clients, or teammates. Detail orientation also matters because small mistakes in software, data, or system maintenance can lead to larger problems. Teamwork appears frequently as well, since many ICT projects require collaboration among developers, analysts, managers, and users.

Overall, we find that employers in Information and Communication value a hybrid candidate profile. They are looking for people who have technical skills, but also strong problem-solving, communication, and teamwork abilities. This shows that ICT jobs are not only about knowing how to use certain software or systems. They are also about applying technical knowledge in real workplace situations. In other words, the ideal ICT candidate is not just technically proficient, but also able to communicate, collaborate, and adapt to organizational needs.

Science and Engineering Professionals

In the Science and Engineering Professionals category, we find that job descriptions are more technical and specialized than the other categories. The language in this section is closely connected to research, engineering, design, testing, analysis, development, and technical problem-solving. Compared with business-related categories, the descriptions here seem to rely more heavily on role-specific knowledge and professional expertise.

The most frequent words point to engineering, research, data, design, systems, analysis, testing, development, and projects. We think this reflects the nature of science and engineering roles, where candidates are expected to work with technical processes, scientific methods, and structured problem-solving. These jobs often require candidates to analyze information, design solutions, test systems, or contribute to research and development work.

Looking at common two-word phrases helps us understand the responsibilities in this category more specifically. Many phrases are connected to technical design, data analysis, project management, quality control, research development, and engineering support. We find this useful because many technical responsibilities are better captured as phrases rather than single words. For example, “data analysis” is more meaningful than looking at “data” and “analysis” separately, and “quality control” points to a specific kind of workplace responsibility.

The TF-IDF comparison also shows that different roles within Science and Engineering have their own distinctive vocabulary. This tells us that the category is not completely uniform. Engineering-related roles may emphasize design, testing, systems, and production, while science-related roles may focus more on research, analysis, laboratory work, or data interpretation. TF-IDF helps reveal these differences because it highlights words that are especially important for certain roles, instead of only showing words that appear often everywhere.

Another part of this section was supervised classification, where we tested whether job labels could be predicted from the text alone. The classification results suggest that many science and engineering roles have clear language patterns. This means the wording of the job descriptions contains enough information to separate many roles from one another. We think this is important because it shows that job descriptions in this category are highly role-specific. Employers use specialized vocabulary to describe the skills, responsibilities, and knowledge required for each position.

At the same time, we realize that accuracy alone is not enough to evaluate the models, especially because some job classes are much smaller than others. This is why macro-averaged F1 score is important. Macro F1 gives equal weight to each class, so it gives a better picture of whether the model performs well across both large and small job categories. The use of cross-validation also makes the results more reliable because the model is tested across different splits of the data instead of only one train-test split.

The topic modeling results give another way to understand the structure of this category. Instead of using existing labels, LDA groups job descriptions based on word co-occurrence patterns. By comparing the topics with major job labels and job titles, we can see whether the text naturally forms themes that match the dataset's occupational categories. We think this is useful because it checks whether the categories are meaningful from the language itself, not only from the labels assigned in the dataset.

Overall, the Science and Engineering results suggest that employers place the strongest emphasis on specialized technical knowledge and role-specific expertise. However, we also find that technical ability is not the only requirement. Terms related to project management, communication, collaboration, and analysis also appear. This suggests that science and engineering candidates are expected to apply their expertise in team-based and organizational settings. The main finding is that this category has the clearest technical vocabulary, but still includes important transferable workplace traits.

Administration and Commercial

For the Administration and Commercial category, we find that employers emphasize leadership, communication, customer service, sales, marketing, organization, and commercial awareness. This category is broad and internally diverse. It includes roles related to finance, marketing, sales, operations, management, and administrative leadership. Because of this diversity, the results show both general managerial traits and more specific commercial responsibilities.

The job title frequency analysis helps us understand the composition of the category. Since Administration and Commercial Managers is not one single occupation, the job titles show that this group includes different types of managerial and commercial positions. Some roles are closer to finance and business control, while others are closer to marketing, sales, customer service, or operations. We think this matters because the language of the job descriptions reflects this mixed structure. A finance-related posting may emphasize accuracy, reporting, and control, while a marketing or sales posting may emphasize customers, communication, growth, and business development.

The trait lexicon analysis connects directly to our research question because it shows which candidate traits appear most often in the job descriptions. The results suggest that communication, leadership, organization, problem-solving, customer orientation, attention to detail, and management ability are especially important in this category. We think these traits make sense for administrative and commercial management roles. Managers are expected to coordinate people, make decisions, communicate goals, oversee operations, and maintain relationships with customers or clients. Commercial roles also require awareness of markets, sales performance, customer needs, and business strategy.

Looking at frequent two-word phrases gives more specific evidence for these patterns. Common phrases include “customer service,” “digital marketing,” “attention to detail,” “sales management,” “business development,” and “project management.” These phrases show that employers emphasize both external-facing and internal-facing responsibilities. “Customer service” and “business development” suggest that firms value candidates who can interact with markets and clients. “Attention to detail” and “project management” suggest that firms also value careful execution and organizational control. From this, we find that Administration and Commercial roles require both strategic thinking and operational ability.

When we focus on adjectives through POS tagging, we can see how employers describe ideal candidates and work expectations. Adjectives in job descriptions often signal valued personal qualities, such as “strong,” “excellent,” “effective,” “organized,” “responsible,” “professional,” and “detail-oriented.” We think these words are important because they show the evaluative language employers use when describing desirable applicants. While nouns often describe tasks or fields, adjectives show how employers imagine the ideal worker. In this category, the adjective

patterns suggest that firms value candidates who appear capable, reliable, organized, and effective in communication and leadership.

The TF-IDF results help identify terms that are especially representative of the Administration and Commercial category. Compared with simple frequency analysis, TF-IDF reduces the importance of generic words and highlights more distinctive vocabulary. These terms are related to management, commercial operations, customers, sales, marketing, financial control, business planning, and organizational responsibility. We think this supports the idea that the category combines managerial oversight with market-facing functions.

The topic model further reveals the internal structure of the category. The main themes include financial control, business management, sales and marketing, and operational services. These topics suggest that Administration and Commercial is not one single type of work, but a collection of related managerial and business functions. The topic model helps explain why the category includes such a wide range of vocabulary. Different subgroups emphasize different responsibilities, but they are connected by a shared focus on organization, leadership, and commercial performance.

Overall, the Administration and Commercial results suggest that employers value candidates who can combine leadership with practical business execution. The strongest pattern is the importance of communication, organization, customer orientation, and management ability. Unlike Science and Engineering, where technical vocabulary is more dominant, Administration and Commercial postings emphasize the ability to manage people, processes, customers, and business goals. We think this category highlights the importance of broad professional traits that can be applied across different business settings.

Overall Comparison Across Categories

Across the four categories, we find that employers do not focus on only one ability when evaluating candidates. Instead, they consider both job-specific skills and general workplace abilities. However, the emphasis changes depending on the type of job. Some categories place more weight on technical knowledge, while others focus more on communication, leadership, management, customer service, or organizational ability.

In Business and Administration, employers emphasize communication, teamwork, management, experience, and organizational support. These roles usually require candidates to coordinate tasks, support daily business operations, and communicate with clients, customers, or internal teams. Although the category includes different directions such as accounting, sales, bookkeeping, and administrative support, the common expectation is that candidates should be reliable, organized, and able to work well with others.

In Information and Communication, job descriptions focus more on technical skills, systems, software, data, and problem-solving. At the same time, communication and teamwork are still frequently mentioned. We think this shows that ICT roles require more than technical knowledge. Candidates also need to solve practical problems, explain technical content, and work with users or team members. These roles combine technical ability with workplace communication skills.

In Science and Engineering Professionals, the technical nature of the jobs is the most obvious. Words related to research, engineering, design, testing, analysis, and development appear frequently. The classification results also suggest that the language used in these job descriptions is relatively distinctive, since many job labels can be predicted from the text alone. However, we also find words related to project management, collaboration, and communication. This means that although science and engineering roles value professional knowledge, candidates still need to apply that knowledge in team and organizational environments.

In Administration and Commercial, employers place more emphasis on leadership, customer service, sales, marketing, organization, and commercial awareness. Compared with the other categories, these roles focus more on management and market-facing ability. Companies want candidates who can manage business processes, communicate with customers, support sales or marketing goals, and contribute to commercial growth.

One major common finding is that communication and problem-solving matter across almost all categories. Even in more technical fields such as Information and Communication and Science and Engineering, employers do not focus only on technical skills. They also emphasize communication, teamwork, and the ability to solve real workplace problems. We think this shows that companies evaluate candidates not only by whether they have a certain skill, but also by whether they can use that skill effectively with other people in a real work environment.

We also realize that different NLP methods help us understand employer preferences from different angles. Word frequency analysis shows the most repeated words. Bigram analysis gives more specific phrases. TF-IDF highlights the most representative words in each category. Trait lexicon analysis directly shows how often different candidate traits are mentioned. POS tagging helps us observe the descriptive language used for ideal candidates. LDA topic modeling shows larger themes and subgroups within each category. By combining these methods, we get a more complete understanding of employer preferences than we would from one method alone.

Overall, our analysis shows that employers have multidimensional expectations for candidates. Technical skills are important, especially in ICT and science-related roles, but general workplace abilities such as communication, teamwork, organization, problem-solving, leadership, and adaptability also appear repeatedly. The priorities differ across job categories, but the overall

trend is clear: the ideal candidate is not only someone who can complete job tasks, but also someone who can communicate, collaborate, adapt, and support the goals of the organization.

5. Conclusion:

This project mainly explores which traits companies value most when evaluating candidates across different job categories. We used the Scraped Job Descriptions dataset from Kaggle and focused on four main categories: Business and Administration, Information and Communication, Science and Engineering, and Administration and Commercial. From the overall results, we find that employers care about both job-specific skills and broader workplace abilities. Even though each field has its own technical requirements, companies still commonly look for candidates who can communicate clearly, solve problems, work with others, and contribute to the organization's goals.

One major thing we realized is that employer preferences are not limited to technical qualifications. In more technical fields, such as Information and Communication and Science and Engineering, job descriptions often mention specialized knowledge, software, systems, research, analysis, design, and development. However, these same categories also repeatedly mention problem-solving, communication, teamwork, and leadership. We think this shows that even technical roles are not only about technical ability. Companies also expect candidates to apply their knowledge in real workplace situations, explain their ideas clearly, and collaborate with different teams.

For the business-related categories, we find that employers place more emphasis on communication, organization, management, customer service, leadership, and commercial awareness. Business and Administration roles usually require candidates to support daily operations, coordinate with others, and handle information or customer-related responsibilities. Administration and Commercial roles focus more on leadership, sales, marketing, customer relationships, and business decision-making. From these results, we think employers in business fields value candidates who can combine interpersonal communication with practical organizational skills.

6. Reference:

Cavaco, M. (n.d.). *Scraped job descriptions* [Data set]. Kaggle.
<https://www.kaggle.com/datasets/marcocavaco/scraped-job-descriptions>

```
import pandas as pd
from google.colab import files

uploaded = files.upload()
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving JD_data.csv to JD_data (1).csv

```
df = pd.read_csv("JD_data.csv")
df.head()
```

Unnamed: 0	ISCO	major_job	job	position	location	description
0	0	21	SCIENCE AND ENGINEERING PROFESSIONALS	physicist	Accelerator Physicist id54315	Villigen PSI, Aargau [' You have an academic degree in physics or e...
1	1	21	SCIENCE AND ENGINEERING PROFESSIONALS	physicist	Applied Physicist (Computing) (EP-LBC-2021-125...	Geneva [' Be in charge of the development of applicat...
2	2	21	SCIENCE AND ENGINEERING PROFESSIONALS	physicist	Accelerator Physicist (BE-ABP-LNO-2021-122-LD)...	Geneva [' Contribute to the maintenance and developme...
3	3	21	SCIENCE AND ENGINEERING PROFESSIONALS	physicist	Medical Devices Physicist	Newton, Cambridgeshire [' Agency: Newton Colmore Consulting Reference...
4	4	21	SCIENCE AND ENGINEERING PROFESSIONALS	physicist	Fluidics Physicist	Cambridge [' Agency: Newton Colmore Consulting Reference...

```
df["major_job"].value_counts()
```

major_job	count
SCIENCE AND ENGINEERING PROFESSIONALS	885
HEALTH PROFESSIONALS	626
INFORMATION AND COMMUNICATIONS TECHNOLOGY PROFESSIONALS	447
ADMINISTRATIVE AND COMMERCIAL MANAGERS	321
BUSINESS AND ADMINISTRATION PROFESSIONALS	288
BUSINESS AND ADMINISTRATION ASSOCIATE PROFESSIONALS	253
TEACHING PROFESSIONALS	252
CUSTOMER SERVICES CLERKS	239
DRIVERS AND MOBILE PLANT OPERATORS	211
CLEANERS AND HELPERS	202
SALES WORKERS	185
MARKET-ORIENTED SKILLED AGRICULTURAL WORKERS	121
PROTECTIVE SERVICES WORKERS	111
SCIENCE AND ENGINEERING ASSOCIATE PROFESSIONALS	104
INFORMATION AND COMMUNICATIONS TECHNICIANS	94
HEALTH ASSOCIATE PROFESSIONALS	73

dtype: int64

```
sorted(df["major_job"].dropna().unique())
```

```
['ADMINISTRATIVE AND COMMERCIAL MANAGERS',
 'BUSINESS AND ADMINISTRATION ASSOCIATE PROFESSIONALS',
 'BUSINESS AND ADMINISTRATION PROFESSIONALS',
 'CLEANERS AND HELPERS',
 'CUSTOMER SERVICES CLERKS',
 'DRIVERS AND MOBILE PLANT OPERATORS',
 'HEALTH ASSOCIATE PROFESSIONALS',
 'HEALTH PROFESSIONALS',
 'INFORMATION AND COMMUNICATIONS TECHNICIANS',
 'INFORMATION AND COMMUNICATIONS TECHNOLOGY PROFESSIONALS',
 'MARKET-ORIENTED SKILLED AGRICULTURAL WORKERS',
 'PROTECTIVE SERVICES WORKERS',
 'SALES WORKERS',
 'SCIENCE AND ENGINEERING ASSOCIATE PROFESSIONALS',
 'SCIENCE AND ENGINEERING PROFESSIONALS',
 'TEACHING PROFESSIONALS']
```

```
business_cats = [
    "BUSINESS AND ADMINISTRATION PROFESSIONALS",
    "BUSINESS AND ADMINISTRATION ASSOCIATE PROFESSIONALS"
]
```

```
biz_df = df[df["major_job"].isin(business_cats)].copy()
```

```
print("Full dataset shape:", df.shape)
print("Business/Admin subset shape:", biz_df.shape)
print()
print(biz_df["major_job"].value_counts())
```

```
Full dataset shape: (4412, 7)
Business/Admin subset shape: (541, 7)
```

```
major_job
BUSINESS AND ADMINISTRATION PROFESSIONALS    288
BUSINESS AND ADMINISTRATION ASSOCIATE PROFESSIONALS  253
Name: count, dtype: int64
```

```
biz_df[["major_job", "job", "position", "location", "description"]].head(10)
```

	major_job	job	position	location	description
885	BUSINESS AND ADMINISTRATION ASSOCIATE PROFESSI...	audit assistant	Audit Assistant Graduate (CA)	Edinburgh	[112 UCAS (new tariff) across 3 A Levels or ...
886	BUSINESS AND ADMINISTRATION ASSOCIATE PROFESSI...	audit assistant	Audit Assistant	Sofia	[За да продължите, е необходимо да потвърдит...
887	BUSINESS AND ADMINISTRATION ASSOCIATE PROFESSI...	audit assistant	Audit Assistant Manager	London	[Identifies risk matters and raises with a m...
888	BUSINESS AND ADMINISTRATION ASSOCIATE PROFESSI...	audit assistant	Privately Owned Business Audit Assistant Manager	United Kingdom	[Any privately-owned business/group by an in...
889	BUSINESS AND ADMINISTRATION ASSOCIATE PROFESSI...	audit assistant	Audit Assistant Manager	United Kingdom	[Assisting and advising the audit team on th...
890	BUSINESS AND ADMINISTRATION ASSOCIATE PROFESSI...	audit assistant	Audit Assistant Manager/Senior Associate	London	[Identifies risk matters and raises with a m...
891	BUSINESS AND ADMINISTRATION ASSOCIATE PROFESSI...	audit assistant	Audit Assistant Manager	Galway	[Reporting into a management team of partner...
892	BUSINESS AND ADMINISTRATION ASSOCIATE PROFESSI...	audit assistant	IT Audit Assistant Manager	Manchester	[Act as a subject matter expert within the c...
893	BUSINESS AND ADMINISTRATION ASSOCIATE PROFESSI...	audit assistant	Royalty Audit Assistant Manager (Happy to talk...	London	[Be responsible for a portfolio of audit cli...
894	BUSINESS AND ADMINISTRATION ASSOCIATE PROFESSI...	audit assistant	Audit Assistant Graduate (ACA)	Manchester	[112 UCAS (new tariff) across 3 A Levels or ...

```
import ast
import re

def clean_description(x):
    if pd.isna(x):
        return ""

    x = str(x)

    # try to convert stringified list into actual text
    try:
        parsed = ast.literal_eval(x)
        if isinstance(parsed, list):
            x = " ".join(str(item) for item in parsed)
    except:
        pass

    # basic cleaning
    x = x.lower()
    x = re.sub(r"http\S+|www\S+", " ", x)
    x = re.sub(r"<.*>", " ", x)
    x = re.sub(r"[^a-z\s]", " ", x)
    x = re.sub(r"\s+", " ", x).strip()

    return x

biz_df["text_clean"] = biz_df["description"].apply(clean_description)
biz_df[["position", "text_clean"]].head()
```

	position	text_clean
885	Audit Assistant Graduate (CA)	ucas new tariff across a levels or an equivale...
886	Audit Assistant	
887	Audit Assistant Manager	identifies risk matters and raises with a mana...
888	Privately Owned Business Audit Assistant Manager	any privately owned business group by an indiv...
889	Audit Assistant Manager	assisting and advising the audit team on the f...

preprocessing tokenization + lemmatization

```
!pip install -q spacy
!python -m spacy download en_core_web_sm
```

Collecting en-core-web-sm==3.8.0
 Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
 12.8/12.8 MB 34.5 MB/s eta 0:00:00

✓ Download and installation successful
 You can now load the package via spacy.load('en_core_web_sm')
 ▲ Restart to reload dependencies
 If you are in a Jupyter or Colab notebook, you may need to restart Python in order to load all the package's dependencies. You can do this by selecting the 'Restart kernel' or 'Restart runtime' option.

```
import spacy

nlp = spacy.load("en_core_web_sm")

def preprocess_text(text):
    doc = nlp(text)
    tokens = [
        token.lemma_
        for token in doc
        if token.is_alpha
        and not token.is_stop
        and len(token.text) > 2
    ]
    return tokens

biz_df["tokens"] = biz_df["text_clean"].apply(preprocess_text)
biz_df[["position", "text_clean", "tokens"]].head(5)
```

	position	text_clean	tokens
885	Audit Assistant Graduate (CA)	ucas new tariff across a levels or an equivale...	[ucas, new, tariff, level, equivalent, level, ...
886	Audit Assistant		[]
887	Audit Assistant Manager	identifies risk matters and raises with a mana...	[identify, risk, matter, raise, manager, partn...
888	Privately Owned Business Audit Assistant Manager	any privately owned business group by an indiv...	[privately, own, business, group, individual, ...
889	Audit Assistant Manager	assisting and advising the audit team on the f...	[assist, advise, audit, team, formulation, aud...

```
print("Empty cleaned texts:", (biz_df["text_clean"] == "").sum())
print("Empty token lists:", biz_df["tokens"].apply(len).eq(0).sum())
```

```
Empty cleaned texts: 3
Empty token lists: 3
```

```
biz_df = biz_df[biz_df["tokens"].apply(len) > 0].copy()
print("Shape after dropping empty token rows:", biz_df.shape)
```

Shape after dropping empty token rows: (538, 9)

▼ frequency analysis

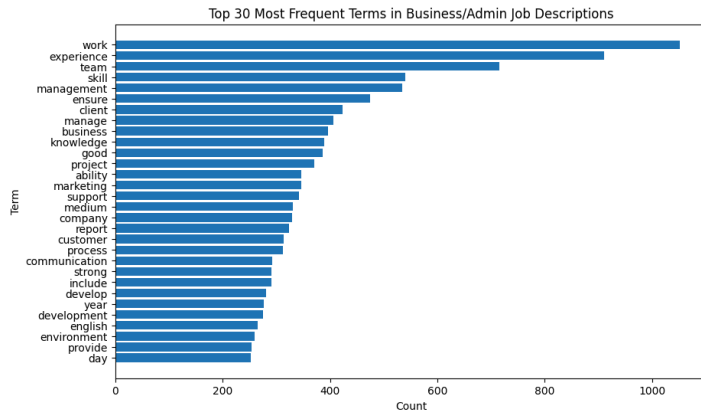
```
from collections import Counter

all_tokens = [token for doc in biz_df["tokens"] for token in doc]
token_counts = Counter(all_tokens)

freq_df = pd.DataFrame(token_counts.most_common(30), columns=["term", "count"])
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.barh(freq_df["term"][:-1], freq_df["count"][:-1])
plt.xlabel("Count")
plt.ylabel("Term")
plt.title("Top 30 Most Frequent Terms in Business/Admin Job Descriptions")
plt.show()
```



▼ stop words

```
custom_stopwords = {
    "work", "experience", "team", "skill", "ensure", "business",
    "knowledge", "good", "ability", "support", "company", "include",
    "provide", "day", "year", "medium", "environment", "development",
    "develop", "strong"
}

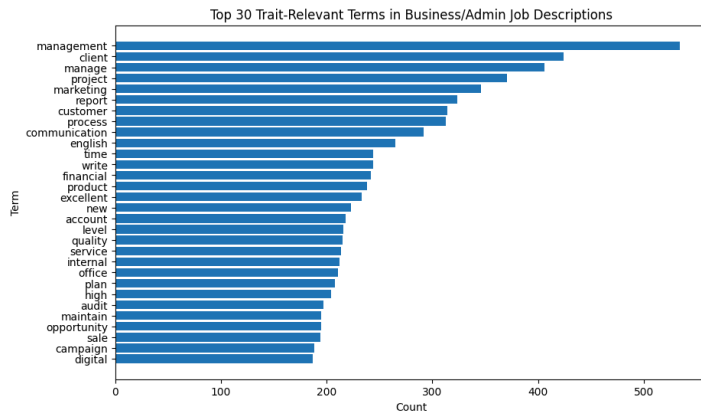
def preprocess_text_custom(text):
    doc = nlp(text)
    tokens = [
        token.lemma_
        for token in doc
        if token.is_alpha
        and not token.is_stop
        and len(token.text) > 2
        and token.lemma_ not in custom_stopwords
    ]
    return tokens

biz_df["tokens_custom"] = biz_df["text_clean"].apply(preprocess_text_custom)

all_tokens_custom = [token for doc in biz_df["tokens_custom"] for token in doc]
token_counts_custom = Counter(all_tokens_custom)

freq_df_custom = pd.DataFrame(
    token_counts_custom.most_common(30),
    columns=["term", "count"]
)
```

```
plt.figure(figsize=(10, 6))
plt.barh(freq_df_custom["term"][:-1], freq_df_custom["count"][:-1])
plt.xlabel("Count")
plt.ylabel("Term")
plt.title("Top 30 Trait-Relevant Terms in Business/Admin Job Descriptions")
plt.show()
```



bigrams

```
print(biz_df.columns.tolist())

['Unnamed: 0', 'ISCO', 'major_job', 'job', 'position', 'location', 'description', 'text_clean', 'tokens']
```

```
biz_df["processed_text"] = biz_df["tokens"].apply(lambda x: " ".join(x))
biz_df[["position", "processed_text"]].head()
```

	position	processed_text
885	Audit Assistant Graduate (CA)	ucas new tariff level equivalent level quali...
887	Audit Assistant Manager	identify risk matter raise manager partner exe...
888	Privately Owned Business Audit Assistant Manager	privately own business group individual family...
889	Audit Assistant Manager	assist advise audit team formulation audit str...
890	Audit Assistant Manager/Senior Associate	identify risk matter raise manager partner exe...

```
from sklearn.feature_extraction.text import CountVectorizer

bigram_vectorizer = CountVectorizer(
    ngram_range=(2, 2),
    min_df=5
)

X_bigrams = bigram_vectorizer.fit_transform(biz_df["processed_text"])
bigram_counts = X_bigrams.sum(axis=0).A1
bigram_terms = bigram_vectorizer.get_feature_names_out()

bigram_df = pd.DataFrame({
    "bigram": bigram_terms,
    "count": bigram_counts
}).sort_values("count", ascending=False)
```

isolate personal trait-like bigrams

```
personal_trait_keywords = [
    "communication", "team", "player", "attention", "detail",
    "leadership", "management", "analytical", "analysis",
    "problem", "solving", "solve", "interpersonal", "organized",
    "organizational", "motivated", "independent", "adaptable",
    "flexible", "creative", "excellent", "time", "multitask"
]
```

qualification trait

```
qualification_keywords = [
    "experience", "year", "degree", "english", "financial",
    "audit", "account", "marketing", "project", "client",
    "customer", "service", "knowledge", "qualification",
    "certification", "office", "digital", "product"
]
```

```
trait_bigram_df = bigram_df[
    bigram_df["bigram"].apply(
        lambda phrase: any(word in phrase.split() for word in personal_trait_keywords)
    )
].copy()

qualification_bigram_df = bigram_df[
    bigram_df["bigram"].apply(
        lambda phrase: any(word in phrase.split() for word in qualification_keywords)
    )
].copy()
```

```
trait_remove = {
    "ability work",
    "skill ability",
    "good knowledge"
}

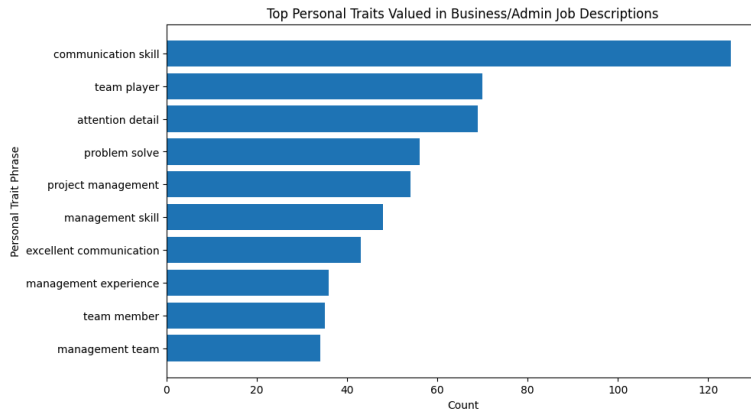
qualification_remove = {
    "competitive salary",
    "social medium"
}

trait_bigram_df = trait_bigram_df[
    ~trait_bigram_df["bigram"].isin(trait_remove)
].copy()
```

```
qualification_bigram_df = qualification_bigram_df[
    ~qualification_bigram_df["bigram"].isin(qualification_remove)
].copy()
```

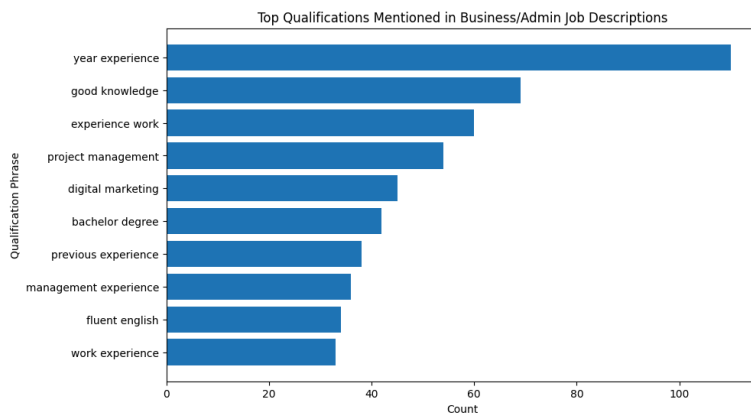
```
top_traits = trait_bigram_df.head(10)
```

```
plt.figure(figsize=(10, 6))
plt.barh(top_traits["bigram"][:-1], top_traits["count"][:-1])
plt.xlabel("Count")
plt.ylabel("Personal Trait Phrase")
plt.title("Top Personal Traits Valued in Business/Admin Job Descriptions")
plt.show()
```



```
top_qual = qualification_bigram_df.head(10)
```

```
plt.figure(figsize=(10, 6))
plt.barh(top_qual["bigram"][:-1], top_qual["count"][:-1])
plt.xlabel("Count")
plt.ylabel("Qualification Phrase")
plt.title("Top Qualifications Mentioned in Business/Admin Job Descriptions")
plt.show()
```



▼ POS / noun phrase extraction

```
def extract_noun_chunks(text):
    doc = nlp(text)
    chunks = []

    for chunk in doc.noun_chunks:
        phrase = " ".join(
            token.lemma_.lower()
            for token in chunk
            if token.is_alpha and not token.is_stop
        ).strip()

        if phrase and len(phrase.split()) <= 4:
            chunks.append(phrase)

    return chunks

biz_df["noun_chunks"] = biz_df["text_clean"].apply(extract_noun_chunks)
biz_df[["position", "noun_chunks"]].head()
```

	position	noun_chunks
885	Audit Assistant Graduate (CA)	[lucas, new tariff, level, equivalent level qua...
887	Audit Assistant Manager	[risk matter, manager, partner, judgement, agr...
888	Privately Owned Business Audit Assistant Manager	[privately own business group, individual s fa...
889	Audit Assistant Manager	[audit team, formulation, audit strategy, plan...
890	Audit Assistant Manager/Senior Associate	[risk matter, manager, partner, judgement, nee...

count the most common noun chunks

```

from collections import Counter

all_chunks = [chunk for row in biz_df["noun_chunks"] for chunk in row]
chunk_counts = Counter(all_chunks)

chunk_df = pd.DataFrame(
    chunk_counts.most_common(50),
    columns=["phrase", "count"]
)

chunk_df.head(20)

```

	phrase	count
0	experience	199
1	client	193
2	ability	175
3	team	145
4	time	102
5	year	99
6	day	92
7	work	91
8	opportunity	81
9	business	81
10	project	80
11	development	80
12	english	76
13	company	75
14	management	69
15	customer	67
16	skill	62
17	knowledge	61
18	compliance	60
19	responsibility	58

▼ split noun chunks into personal traits vs qualifications

```

personal_trait_keywords = [
    "communication", "detail", "team", "player", "leadership",
    "management", "problem", "time", "interpersonal",
    "analytical", "organization", "organisational",
    "organizational", "motivation", "initiative",
    "adaptability", "flexibility", "creativity"
]

```

```

qualification_keywords = [
    "experience", "degree", "english", "finance", "financial",
    "audit", "account", "marketing", "project", "client",
    "customer", "service", "knowledge", "office", "digital",
    "product", "qualification", "certificate", "background"
]

```

```

trait_chunk_df = chunk_df[
    chunk_df["phrase"].apply(
        lambda phrase: any(word in phrase.split() for word in personal_trait_keywords)
    )
].copy()

qualification_chunk_df = chunk_df[
    chunk_df["phrase"].apply(
        lambda phrase: any(word in phrase.split() for word in qualification_keywords)
    )
].copy()

print("Trait-like noun chunks:")
display(trait_chunk_df.head(20))

print("Qualification-like noun chunks:")
display(qualification_chunk_df.head(20))

```

Trait-like noun chunks:

	phrase	count
3	team	145
4	time	102
14	management	69
22	detail	54
39	initiative	40

Qualification-like noun chunks:

	phrase	count
0	experience	199
1	client	193
10	project	80
12	english	76
15	customer	67
17	knowledge	61
28	service	51
30	good knowledge	49
44	product	38

```

trait_chunk_remove = set()

qualification_chunk_remove = {
    "competitive salary", "new opportunity"
}

```

```

trait_chunk_df = trait_chunk_df[
    ~trait_chunk_df["phrase"].isin(trait_chunk_remove)
].copy()

qualification_chunk_df = qualification_chunk_df[
    ~qualification_chunk_df["phrase"].isin(qualification_chunk_remove)
].copy()

print("Cleaned trait noun chunks:")
display(trait_chunk_df.head(15))

print("Cleaned qualification noun chunks:")
display(qualification_chunk_df.head(15))

```

Cleaned trait noun chunks:

	phrase	count
3	team	145
4	time	102
14	management	69
22	detail	54
39	initiative	40

Cleaned qualification noun chunks:

	phrase	count
0	experience	199
1	client	193
10	project	80
12	english	76
15	customer	67
17	knowledge	61
28	service	51
30	good knowledge	49
44	product	38

Personal traits graph

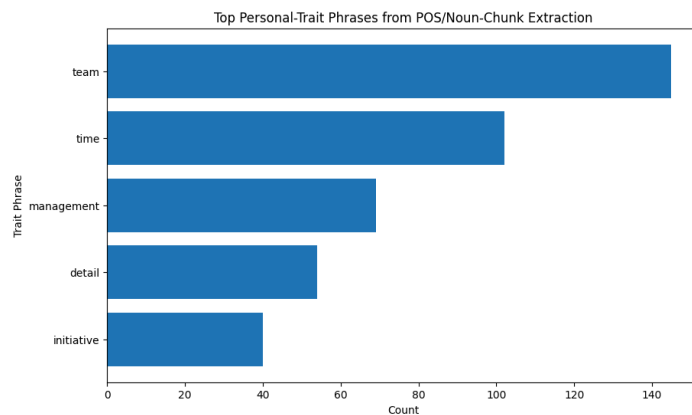
```

import matplotlib.pyplot as plt

top_trait_chunks = trait_chunk_df.head(10)

plt.figure(figsize=(10, 6))
plt.barh(top_trait_chunks["phrase"][:-1], top_trait_chunks["count"][:-1])
plt.xlabel("Count")
plt.ylabel("Trait Phrase")
plt.title("Top Personal-Trait Phrases from POS/Noun-Chunk Extraction")
plt.show()

```



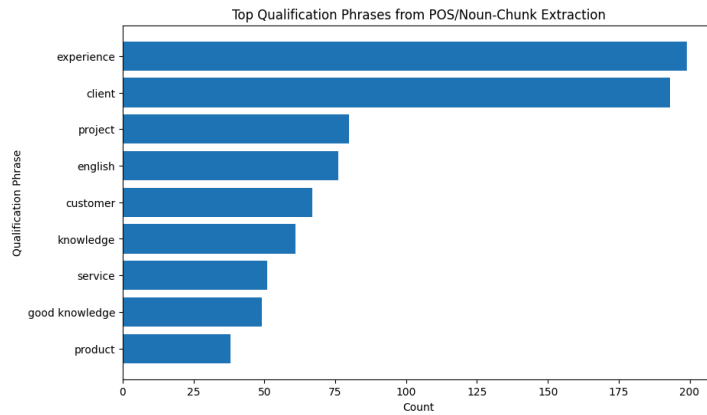
Qualifications graph

```

top_qualification_chunks = qualification_chunk_df.head(10)

plt.figure(figsize=(10, 6))
plt.barh(top_qualification_chunks["phrase"][:-1], top_qualification_chunks["count"][:-1])
plt.xlabel("Count")
plt.ylabel("Qualification Phrase")
plt.title("Top Qualification Phrases from POS/Noun-Chunk Extraction")
plt.show()

```



▼ compare business/admin to all other jobs with TF-IDF

```
def preprocess_text_full(text):
    doc = nlp(text)
    tokens = [
        token.lemma_
        for token in doc
        if token.is_alpha
        and not token.is_stop
        and len(token.text) > 2
    ]
    return tokens

df["tokens_full"] = df["text_clean"].apply(preprocess_text_full)
df = df[df["tokens_full"].apply(len) > 0].copy()
print(df.shape)
df["processed_text_full"] = df["tokens_full"].apply(lambda x: " ".join(x))
df[["text_clean", "processed_text_full"]].head()
```

Rebuild the processed full-text TF-IDF input from scratch

```
business_cats = [
    "BUSINESS AND ADMINISTRATION PROFESSIONALS",
    "BUSINESS AND ADMINISTRATION ASSOCIATE PROFESSIONALS"
]

def preprocess_text_full(text):
    doc = nlp(text)
    tokens = [
        token.lemma_.lower()
        for token in doc
        if token.is_alpha
        and not token.is_stop
        and len(token.text) > 2
    ]
    return tokens

# start from the original cleaned full dataframe
df["tokens_full"] = df["text_clean"].apply(preprocess_text_full)
df_full_proc = df[df["tokens_full"].apply(len) > 0].copy()

df_full_proc["processed_text_full"] = df_full_proc["tokens_full"].apply(lambda x: " ".join(x))
df_full_proc["group"] = df_full_proc["major_job"].apply(
    lambda x: "business_admin" if x in business_cats else "other"
)

grouped_df = (
    df_full_proc
    .groupby("group")["processed_text_full"]
    .apply(lambda x: " ".join(x))
    .reset_index()
)

grouped_df
```

	group	processed_text_full
0	business_admin	ucas new tariff level equivalent level qualifi...
1	other	academic degree physics engineering profound k...

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(
    ngram_range=(1, 2),
    min_df=2
)

X_tfidf = tfidf.fit_transform(grouped_df["processed_text_full"])
terms = tfidf.get_feature_names_out()

tfidf_df = pd.DataFrame(
    X_tfidf.toarray(),
    index=grouped_df["group"],
    columns=terms
).T

tfidf_df["difference"] = tfidf_df["business_admin"] - tfidf_df["other"]

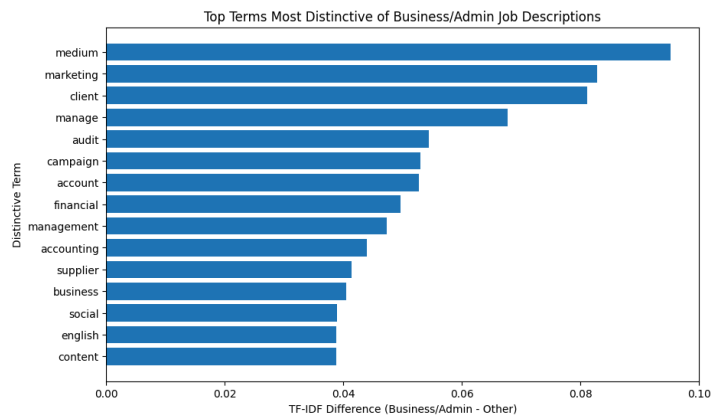
business_distinctive_df = tfidf_df.sort_values("difference", ascending=False)
business_distinctive_df.head(5)
```

group	business_admin	other	difference
medium	0.105317	0.010041	0.095276
marketing	0.110090	0.027305	0.082784
client	0.134907	0.053788	0.081119
manage	0.129180	0.061422	0.067758
audit	0.062681	0.008280	0.054401

```
top_distinctive = business_distinctive_df.head(15).reset_index()
top_distinctive.columns = ["term", "business_admin_tfidf", "other_tfidf", "difference"]
```

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(10, 6))
plt.barh(top_distinctive["term"][:-1], top_distinctive["difference"][:-1])
plt.xlabel("TF-IDF Difference (Business/Admin - Other)")
plt.ylabel("Distinctive Term")
plt.title("Top Terms Most Distinctive of Business/Admin Descriptions")
plt.show()
```



module 9 LDA Topic Modeling

```
biz_df["lda_text"] = biz_df["tokens"].apply(lambda x: " ".join(x))
biz_df[["position", "lda_text"]].head()
```

	position	lda_text
885	Audit Assistant Graduate (CA)	ucas new tariff level equivalent level quali...
887	Audit Assistant Manager	identify risk matter raise manager partner exe...
888	Privately Owned Business Audit Assistant Manager	privately own business group individual family...
889	Audit Assistant Manager	assist advise audit team formulation audit str...
890	Audit Assistant Manager/Senior Associate	identify risk matter raise manager partner exe...

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
lda_vectorizer = CountVectorizer(
    max_df=0.95,
    min_df=5,
    stop_words="english"
)

lda_dtm = lda_vectorizer.fit_transform(biz_df["lda_text"])
lda_terms = lda_vectorizer.get_feature_names_out()

print(lda_dtm.shape)

(538, 1676)
```

```
from sklearn.decomposition import LatentDirichletAllocation
```

```
lda_model = LatentDirichletAllocation(
    n_components=4,
    random_state=42
)

lda_model.fit(lda_dtm)
```

```
LatentDirichletAllocation(n_components=4, random_state=42)
```

```
def print_topics(model, feature_names, n_top_words=10):
    for topic_idx, topic in enumerate(model.components_):
        top_features = topic.argsort()[::-n_top_words:]
        top_words = [feature_names[i] for i in top_features]
        print(f"Topic {topic_idx + 1}: {' '.join(top_words)}")

print_topics(lda_model, lda_terms, n_top_words=10)
```

```
Topic 1: ensure, work, audit, experience, quality, standard, client, inspection, team, management
Topic 2: work, team, experience, medium, marketing, client, skill, campaign, digital, social
Topic 3: account, accounting, report, manage, financial, experience, management, work, french, knowledge
Topic 4: experience, work, business, skill, management, team, ensure, supplier, process, strong
```

```
topic_probs = lda_model.transform(lda_dtm)
biz_df["topic"] = topic_probs.argmax(axis=1) + 1

biz_df[["position", "topic"]].head(10)
```

	position	topic
885	Audit Assistant Graduate (CA)	4
887	Audit Assistant Manager	1
888	Privately Owned Business Audit Assistant Manager	1
889	Audit Assistant Manager	1
890	Audit Assistant Manager/Senior Associate	1
891	Audit Assistant Manager	1
892	IT Audit Assistant Manager	1
893	Royalty Audit Assistant Manager (Happy to talk...	1
894	Audit Assistant Graduate (ACA)	4
895	Audit Assistant Manager (Happy to talk flexib...	1

```
# topic sizes
biz_df["topic"].value_counts().sort_index()
```

	count
topic	
1	90
2	203
3	83
4	162

```
dtype: int64
```

```
for t in sorted(biz_df["topic"].unique()):
    print(f"\n==== Topic {t} =====")
    display(biz_df[biz_df["topic"] == t][["position", "job", "location"]].head(10))
```

```
==== Topic 1 =====
```

	position	job	location
887	Audit Assistant Manager	audit assistant	London
888	Privately Owned Business Audit Assistant Manager	audit assistant	United Kingdom
889	Audit Assistant Manager	audit assistant	United Kingdom
890	Audit Assistant Manager/Senior Associate	audit assistant	London
891	Audit Assistant Manager	audit assistant	Galway
892	IT Audit Assistant Manager	audit assistant	Manchester
893	Royalty Audit Assistant Manager (Happy to talk...	audit assistant	London
895	Audit Assistant Manager (Happy to talk flexib...	audit assistant	Glasgow
930	Client Bookkeeper	bookkeeper	Nicosia
936	Inspection Engineer	inspection engineer	Aberdeen

```
==== Topic 2 =====
```

	position	job	location
993	Quality Inspector	inspector	Telford, Shropshire
1004	Vehicle Inspector	inspector	Wakefield
1006	Graduate Field Sales Representative	field sales representative	Leeds
1007	Field Sales Representative	field sales representative	Wicklow
1008	Field Sales Representative	field sales representative	Ireland
1009	Field Sales Representative	field sales representative	Ireland
1010	Field Sales Representative	field sales representative	Wakefield
1011	Field Sales Representative	field sales representative	United Kingdom
1014	Field Sales Representative	field sales representative	Wakefield
1017	Graduate Field Sales Representative	field sales representative	Nottingham

```
==== Topic 3 =====
```

	position	job	location
899	Senior Bookkeeper	bookkeeper	Burgess Hill, West Sussex
902	Bookkeeper	bookkeeper	Oakham, Rutland
903	Senior Bookkeeper	bookkeeper	London
904	Interim Bookkeeper	bookkeeper	London
906	Trainee Bookkeeper	bookkeeper	Tonbridge, Kent
907	Bookkeeper	bookkeeper	Liverpool
909	Bookkeeper	bookkeeper	Sheffield
910	Accounts Assistant/ Bookkeeper	bookkeeper	Wallingford, Oxfordshire
911	Part Time Payroller/Bookkeeper	bookkeeper	Wimborne Minster, Dorset
913	Bookkeeper	bookkeeper	Hove, East Sussex

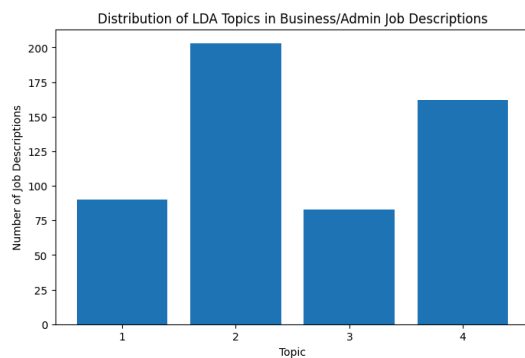
```
==== Topic 4 =====
```

	position	job	location
885	Audit Assistant Graduate (CA)	audit assistant	Edinburgh
894	Audit Assistant Graduate (ACA)	audit assistant	Manchester
896	Assistant Bookkeeper	bookkeeper	Deptford, South East London
897	Assistant Bookkeeper	bookkeeper	Deptford, South East London
898	Bookkeeper	bookkeeper	Wokingham, Berkshire
900	Bookkeeper / Finance Assistant	bookkeeper	Cambridge
901	New Bookkeeper €30-€36k Kildare	bookkeeper	Kildare
905	Senior Bookkeeper	bookkeeper	Malton, North Yorkshire
908	Bookkeeper	bookkeeper	Wokingham, Berkshire
912	Bookkeeper	bookkeeper	London

```
import matplotlib.pyplot as plt

topic_counts = biz_df["topic"].value_counts().sort_index()

plt.figure(figsize=(8, 5))
plt.bar(topic_counts.index.astype(str), topic_counts.values)
plt.xlabel("Topic")
plt.ylabel("Number of Job Descriptions")
plt.title("Distribution of LDA Topics in Business/Admin Job Descriptions")
plt.show()
```



```
for t in sorted(biz_df["topic"].unique()):
    print(f"\n==== Topic {t}: top job labels =====")
    display(biz_df[biz_df["topic"] == t]["job"].value_counts().head(15))
```

```
==== Topic 1: top job labels =====
```

count	
job	
inspector	26
auditor	24
inspection engineer	17
audit assistant	8
manager	8
administrative assistant	2
conference manager	2
recruitment officer	2
bookkeeper	1

```
dtype: int64
```

```
==== Topic 2: top job labels =====
```

count	
job	
copywriter	38
media planner	36
marketer	32
recruitment researcher	20
manager	12
administrative assistant	11
media consultant	11
field sales representative	10
accountant	9
recruitment officer	7
financial analyst	5
buyer	4
auditor	4
conference manager	2
inspector	2

```
dtype: int64
```

```
!pip install gensim -q
```

27.9/27.9 MB 22.4 MB/s eta 0:00:00

Installation and Downloading

```
import pandas as pd
import numpy as np
import re
import ast
from collections import Counter, defaultdict

import matplotlib.pyplot as plt

import nltk
for pkg in [
    "punkt", "punkt_tab", "stopwords", "wordnet", "omw-1.4",
    "averaged_perceptron_tagger", "averaged_perceptron_tagger_eng",
    "vader_lexicon"
]:
    try:
        nltk.download(pkg)
    except:
        pass

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer, PorterStemmer
from nltk import pos_tag
from nltk.sentiment import SentimentIntensityAnalyzer

from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import LatentDirichletAllocation

from gensim.models import Word2Vec
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data] Package omw-1.4 is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data] date!
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
[nltk_data] Package vader_lexicon is already up-to-date!
```

Read data

```
df = pd.read_csv("1.csv", encoding="latin1")

print(df.shape)
print(df.columns.tolist())
df.head()
```

```
(4412, 7)
['Unnamed: 0', 'ISCO', 'major_job', 'job', 'position', 'location', 'description']
```

	Unnamed: 0	ISCO	major_job	job	position	location	description
0	0	21	SCIENCE AND ENGINEERING PROFESSIONALS	physicist	Accelerator Physicist id54315	Villigen PSI, Aargau	[' You have an academic degree in physics or e...
1	1	21	SCIENCE AND ENGINEERING PROFESSIONALS	physicist	Applied Physicist (Computing) (EP-LBC-2021-125...	Geneva	[' Be in charge of the development of applicat...
			SCIENCE AND ENGINEERING		Accelerator Physicist (EP		[' Contribute to the

```
df_admin = df[df["major_job"] == "ADMINISTRATIVE AND COMMERCIAL MANAGERS"].copy()

print("Administrative subset :", df_admin.shape)
print(df_admin["job"].value_counts().head(20))
```

```
Administrative subset : (321, 7)
job
chief accountant      39
chief financial officer 36
cleaning manager      33
product manager       29
sales supervisor      27
marketing manager     26
marketing director    18
administrative manager 14
sales manager         14
sales director        14
laboratory manager    14
environmental manager 13
planning director     11
staff officer         11
research manager      11
export sales manager  11
Name: count, dtype: int64
```

Text preprocessing

```
def parse_description(x):
    """
    description
    """
    if pd.isna(x):
        return ""
    x = str(x)
    try:
        obj = ast.literal_eval(x)
        if isinstance(obj, list):
            return " ".join([str(i) for i in obj])
        return str(obj)
    except:
        return x

def basic_clean(text):
    text = str(text).lower()
    text = re.sub(r"http\S+|www\S+", "", text)
    text = re.sub(r"[a-z\s]", "", text)
    text = re.sub(r"\s+", "", text).strip()
    return text
```

```
df_admin["text_raw"] = df_admin["description"].apply(parse_description)
df_admin["text_clean"] = df_admin["text_raw"].apply(basic_clean)

df_admin[["position", "text_raw", "text_clean"]].head(2)
```

	position	text_raw	text_clean
3889	Chief Accountant	Accountancy A-Z of several companies Contact ...	accountancy a z of several companies contact w...
3890	chief accountant	Accountancy A-Z of several companies Contact ...	accountancy a z of several companies contact w...

```
stop_words = set(stopwords.words("english"))
```

```

custom_stopwords = {
    "experience", "work", "working", "ability", "abilities", "skill", "skills",
    "knowledge", "requirement", "requirements", "required", "candidate", "role",
    "position", "job", "good", "excellent", "strong", "company", "responsibility",
    "responsibilities", "must", "ensure", "including", "across", "within", "etc"
}

all_stopwords = stop_words.union(custom_stopwords)

lemmatizer = WordNetLemmatizer()
stemmer = PorterStemmer()

```

```

def tokenize_lemmatize(text):
    tokens = word_tokenize(text)
    tokens = [t for t in tokens if t.isalpha()]
    tokens = [t for t in tokens if t not in all_stopwords and len(t) > 2]
    lemmas = [lemmatizer.lemmatize(t) for t in tokens]
    return lemmas

def tokenize_stem(text):
    tokens = word_tokenize(text)
    tokens = [t for t in tokens if t.isalpha()]
    tokens = [t for t in tokens if t not in all_stopwords and len(t) > 2]
    stems = [stemmer.stem(t) for t in tokens]
    return stems

```

```

df_admin["tokens_lemma"] = df_admin["text_clean"].apply(tokenize_lemmatize)
df_admin["tokens_stem"] = df_admin["text_clean"].apply(tokenize_stem)
df_admin["text_lemma"] = df_admin["tokens_lemma"].apply(lambda x: " ".join(x))

df_admin[["position", "tokens_lemma", "tokens_stem"]].head(3)

```

	position	tokens_lemma	tokens_stem
3889	Chief Accountant	[accountancy, several, company, contact, suppl...	[account, sever, compani, contact, supplier, c...
3890	chief accountant	[accountancy, several, company, contact, suppl...	[account, sever, compani, contact, supplier, c...
3891	Chief Accountant	[accountancy, several, company, contact, suppl...	[account, sever, compani, contact, supplier, c...

Basic descriptive statistics

```

df_admin["word_count"] = df_admin["tokens_lemma"].apply(len)

print(df_admin["word_count"].describe())

```

```

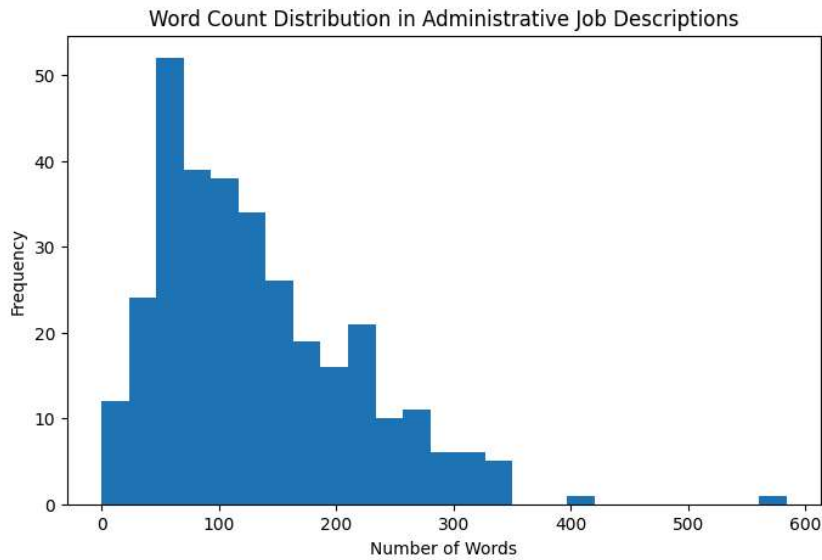
count    321.000000
mean     132.239875
std       84.380436
min        0.000000
25%       66.000000
50%      113.000000
75%      182.000000
max      584.000000
Name: word_count, dtype: float64

```

```

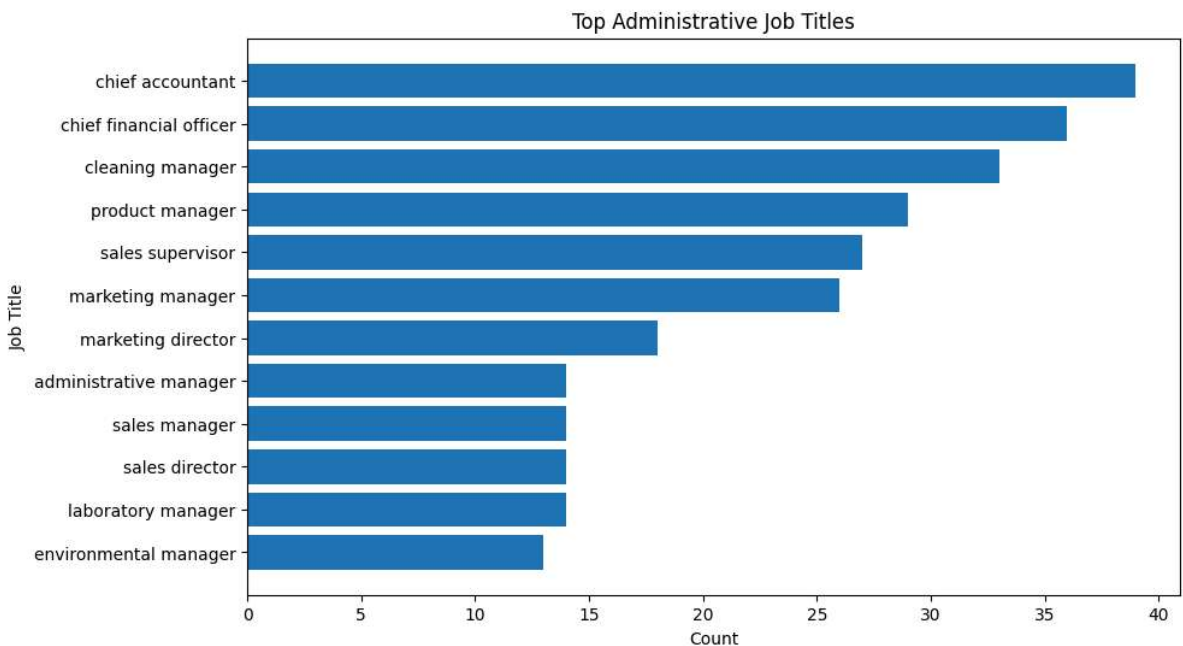
plt.figure(figsize=(8,5))
plt.hist(df_admin["word_count"], bins=25)
plt.title("Word Count Distribution in Administrative Job Descriptions")
plt.xlabel("Number of Words")
plt.ylabel("Frequency")
plt.show()

```



```
top_jobs = df_admin["job"].value_counts().head(12)

plt.figure(figsize=(10,6))
plt.barh(top_jobs.index[:-1], top_jobs.values[:-1])
plt.title("Top Administrative Job Titles")
plt.xlabel("Count")
plt.ylabel("Job Title")
plt.show()
```



Trait dictionary

```
trait_patterns = {
    "communication": r"\b(?:communication|communicate|presentation|present|liaison|stakeholder|interpersonal|negotiat\w*)\b",
    "leadership": r"\b(?:leadership|lead|leading|supervis\w*|manage team|mentor|coach\w*|people management)\b",
    "teamwork": r"\b(?:teamwork|team player|collaborat\w*|cross functional|cooperat\w*|work in team|working in teams)\b",
    "organization": r"\b(?:organiz\w*|planning|coordinate\w*|scheduling|multitask\w*|prioriti\w*)\b",
    "problem_solving": r"\b(?:problem solving|solve problem\w*|troubleshoot\w*|resolve issue\w*|solution orient\w*|solving problem)\b",
    "analytical": r"\b(?:analytical|analysis|analy[sz]e|reporting|forecasting|budgeting|data analysis)\b",
    "detail_oriented": r"\b(?:detail oriented|attention to detail|accuracy|accurate|precise)\b",
}
```

```

"adaptability": r"\b(?:adaptab\w*|flexib\w*|dynamic environment|fast paced|openness to new ideas)\b",
"initiative": r"\b(?:proactive|self starter|self motivated|independent|initiative)\b",
"customer_focus": r"\b(?:customer service|customer focused|client oriented|service oriented|client focus|customer focus)\b",
"project_management": r"\b(?:project management|project manager|project planning|project coordination)\b",
"finance_reporting": r"\b(?:finance|financial|accounting|budget|audit|payroll|tax|reporting|kpi|dashboard|excel|sap|erp)\b",
"sales_marketing": r"\b(?:sales|marketing|brand|promotion|business development|market research)\b",
"compliance_risk": r"\b(?:compliance|regulation|policy|risk management|legal|quality assurance)\b"
}

```

```

for trait, pattern in trait_patterns.items():
    df_admin[f"{trait}"] = df_admin["text_clean"].str.contains(pattern, regex=True).astype(int)

```

```
trait_cols = [f"{t}" for t in trait_patterns.keys()]
```

```

trait_summary = (
    df_admin[trait_cols]
    .mean()
    .sort_values(ascending=False)
    .mul(100)
    .round(2)
)

```

```
trait_summary
```

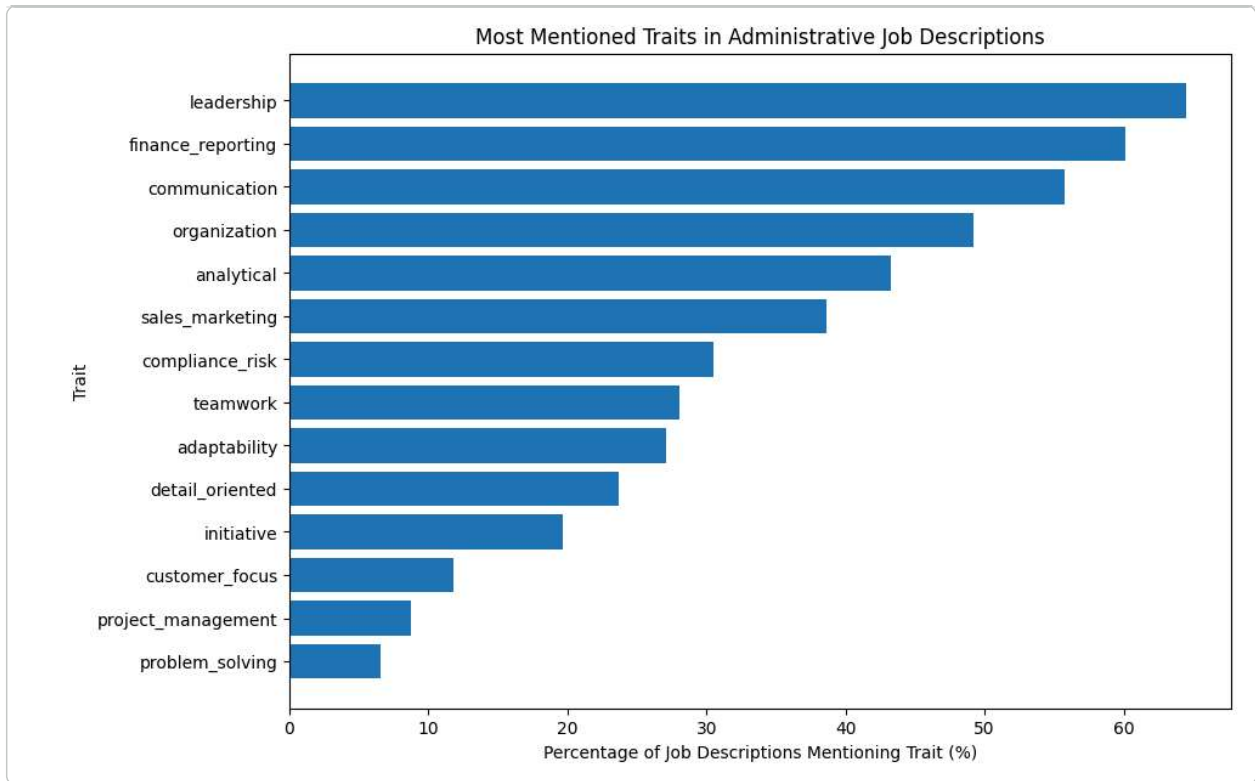
	0
leadership	64.49
finance_reporting	60.12
communication	55.76
organization	49.22
analytical	43.30
sales_marketing	38.63
compliance_risk	30.53
teamwork	28.04
adaptability	27.10
detail_oriented	23.68
initiative	19.63
customer_focus	11.84
project_management	8.72
problem_solving	6.54

```
dtype: float64
```

```

plt.figure(figsize=(10,7))
plt.barh(trait_summary.index[::-1], trait_summary.values[::-1])
plt.title("Most Mentioned Traits in Administrative Job Descriptions")
plt.xlabel("Percentage of Job Descriptions Mentioning Trait (%)")
plt.ylabel("Trait")
plt.show()

```



Compare traits based on specific positions

```
top_job_list = df_admin["job"].value_counts().head(8).index.tolist()
df_top_jobs = df_admin[df_admin["job"].isin(top_job_list)].copy()
```

```
job_trait_compare = (
    df_top_jobs.groupby("job")[trait_cols]
    .mean()
    .mul(100)
    .round(1)
)
```

```
job_trait_compare
```

	communication	leadership	teamwork	organization	problem_solving	analytical	detail_oriented	adaptability
job								
administrative manager	35.7	71.4	21.4	64.3	21.4	42.9	28.6	35.7
chief accountant	23.1	79.5	10.3	33.3	7.7	41.0	25.6	12.8
chief financial officer	61.1	77.8	16.7	69.4	11.1	83.3	16.7	27.8
cleaning manager	72.7	78.8	0.0	45.5	0.0	0.0	48.5	63.6
marketing director	77.8	83.3	61.1	50.0	0.0	66.7	5.6	27.8

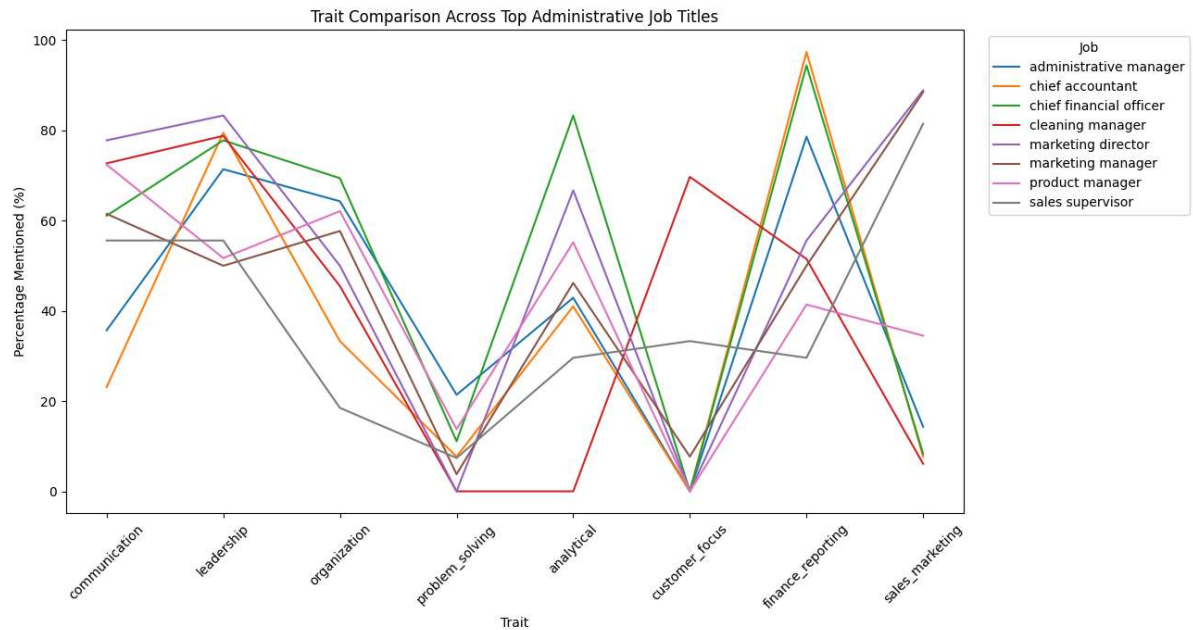
```
selected_traits = [
    "communication",
    "leadership",
    "organization",
    "problem_solving",
    "analytical",
    "customer_focus",
    "finance_reporting",
    "sales_marketing"
]
```

```

]

job_trait_compare[selected_traits].T.plot(figsize=(13,7))
plt.title("Trait Comparison Across Top Administrative Job Titles")
plt.xlabel("Trait")
plt.ylabel("Percentage Mentioned (%)")
plt.xticks(rotation=45)
plt.legend(title="Job", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()

```



TF-IDF

Frequent phrases

```

bigram_vectorizer = CountVectorizer(
    ngram_range=(2, 2),
    min_df=3,
    max_df=0.8
)

X_bigram = bigram_vectorizer.fit_transform(df_admin["text_lemma"])
bigram_freq = np.asarray(X_bigram.sum(axis=0)).ravel()

bigram_df = pd.DataFrame({
    "bigram": bigram_vectorizer.get_feature_names_out(),
    "freq": bigram_freq
}).sort_values("freq", ascending=False)

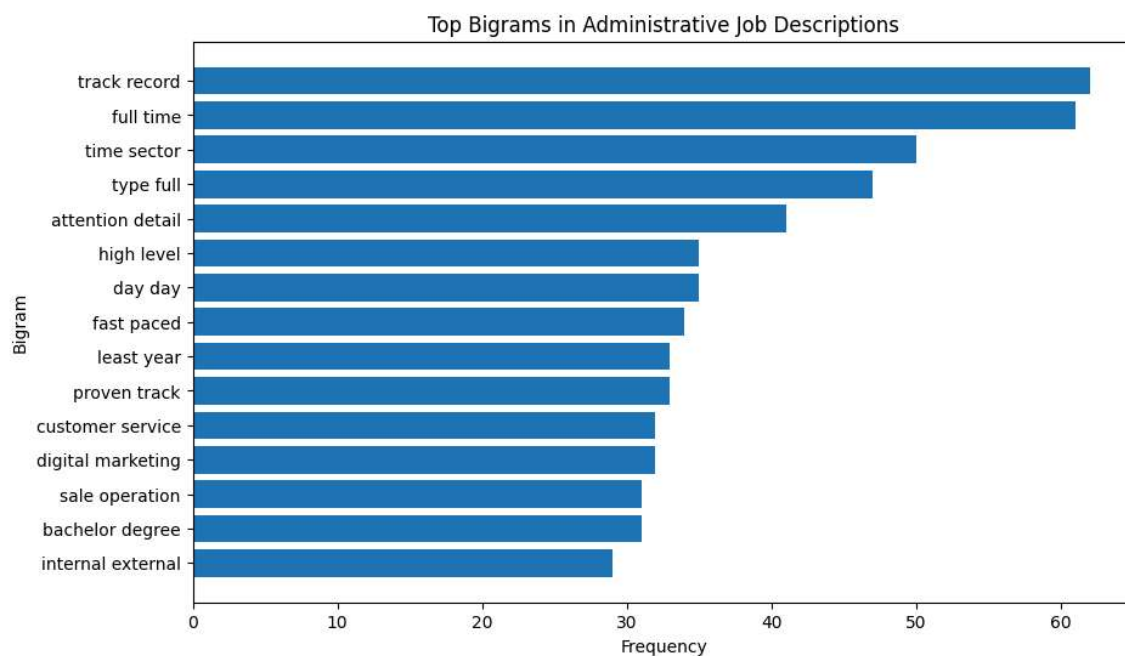
bigram_df.head(20)

```

	bigram	freq
2317	track record	62
923	full time	61
2297	time sector	50
2346	type full	47
171	attention detail	41
1003	high level	35
559	day day	35
821	fast paced	34
1199	least year	33
1779	proven track	33
542	customer service	32
645	digital marketing	32
1964	sale operation	31
193	bachelor degree	31
1101	internal external	29
1764	project management	29
1659	policy procedure	28
1509	operation location	28
1987	sector sale	28
1591	people management	28

```
top_bigrams = bigram_df.head(15)
```

```
plt.figure(figsize=(10,6))
plt.barh(top_bigrams["bigram"][:-1], top_bigrams["freq"][:-1])
plt.title("Top Bigrams in Administrative Job Descriptions")
plt.xlabel("Frequency")
plt.ylabel("Bigram")
plt.show()
```



TF-IDF Keywords

```
tfidf_vectorizer = TfidfVectorizer(
    ngram_range=(1, 2),
    min_df=3,
    max_df=0.75
)

X_tfidf = tfidf_vectorizer.fit_transform(df_admin["text_lemma"])
tfidf_scores = np.asarray(X_tfidf.mean(axis=0)).ravel()

tfidf_df = pd.DataFrame({
    "term": tfidf_vectorizer.get_feature_names_out(),
    "avg_tfidf": tfidf_scores
}).sort_values("avg_tfidf", ascending=False)

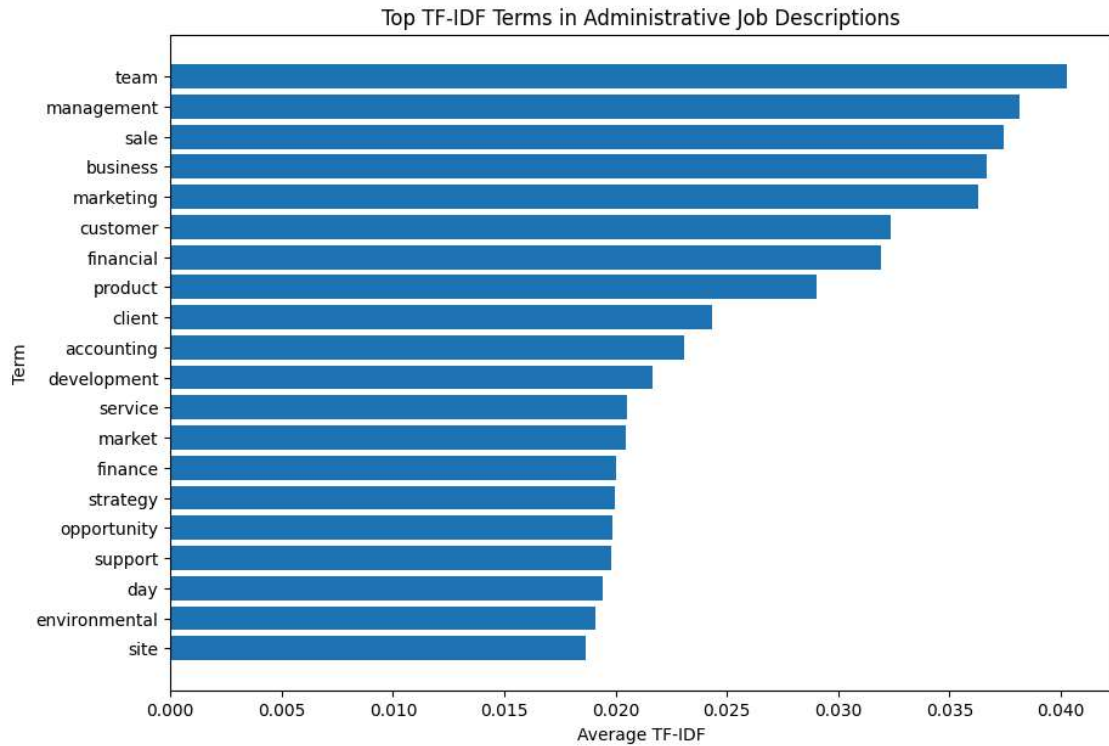
tfidf_df.head(30)
```

	term	avg_tfidf
3989	team	0.040282
2380	management	0.038161
3529	sale	0.037457
512	business	0.036682
2463	marketing	0.036296
981	customer	0.032354
1571	financial	0.031923
3135	product	0.029031
661	client	0.024353
56	accounting	0.023100
1157	development	0.021684
3623	service	0.020507
2443	market	0.020456
1557	finance	0.020012
3846	strategy	0.019941
2745	opportunity	0.019883
3913	support	0.019782
1030	day	0.019425
1399	environmental	0.019097
3682	site	0.018656
3121	process	0.018200
1386	environment	0.018035
2247	level	0.018026
3184	project	0.017986
2372	manage	0.017153
4396	year	0.017048
2715	office	0.016990
3370	relationship	0.016800
2672	new	0.016726
2424	managing	0.016719

```
top_tfidf = tfidf_df.head(20)

plt.figure(figsize=(10, 7))
plt.barh(top_tfidf["term"][:-1], top_tfidf["avg_tfidf"][:-1])
plt.title("Top TF-IDF Terms in Administrative Job Descriptions")
```

```
plt.xlabel("Average TF-IDF")
plt.ylabel("Term")
plt.show()
```



POS tagging

```
def get_pos_lists(tokens):
    tagged = pos_tag(tokens)
    nouns = [w for w, tag in tagged if tag.startswith("NN")]
    adjs = [w for w, tag in tagged if tag.startswith("JJ")]
    verbs = [w for w, tag in tagged if tag.startswith("VB")]
    return nouns, adjs, verbs
```

```
noun_counter = Counter()
adj_counter = Counter()
verb_counter = Counter()

for tokens in df_admin["tokens_lemma"]:
    nouns, adjs, verbs = get_pos_lists(tokens)
    noun_counter.update(nouns)
    adj_counter.update(adjs)
    verb_counter.update(verbs)

print("Top nouns:")
print(noun_counter.most_common(20))

print("\nTop adjectives:")
print(adj_counter.most_common(20))

print("\nTop verbs:")
print(verb_counter.most_common(20))
```

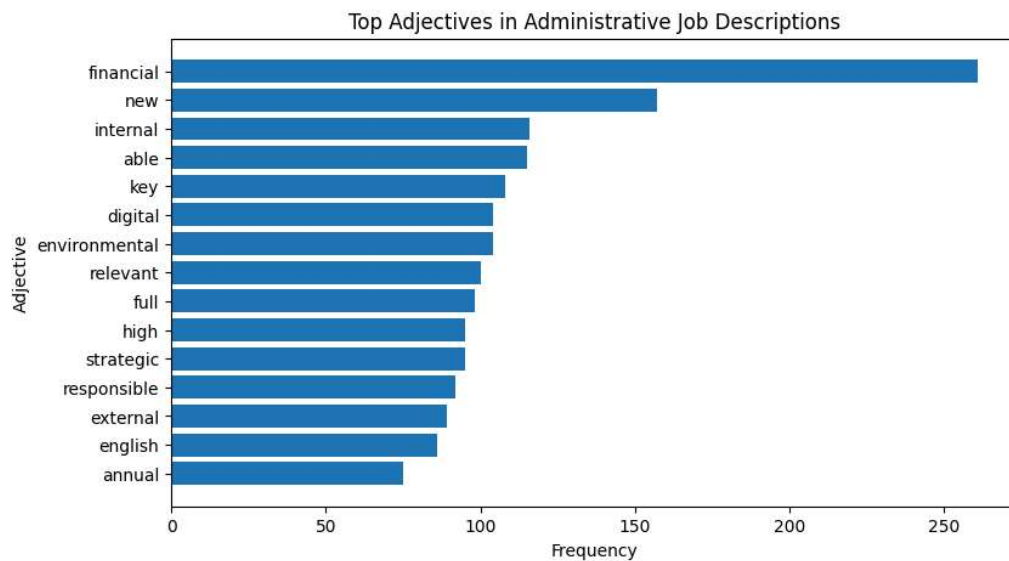
```
Top nouns:
[('team', 558), ('management', 479), ('business', 444), ('customer', 365), ('sale', 347), ('marketing', 289), ('product', 259), ('client', 200), ('development', 190), ('financial', 180), ('service', 170), ('market', 160), ('finance', 150), ('strategy', 140), ('opportunity', 130), ('support', 120), ('day', 110), ('environmental', 100), ('site', 90)]

Top adjectives:
[('financial', 261), ('new', 157), ('internal', 116), ('able', 115), ('key', 108), ('digital', 104), ('environmental', 104), ('relevant', 100), ('strategic', 90), ('business', 80), ('customer', 70), ('product', 60), ('client', 50), ('development', 40), ('financial', 30), ('service', 20), ('market', 10), ('finance', 10), ('strategy', 10)]

Top verbs:
[('develop', 130), ('managing', 116), ('ensuring', 92), ('based', 90), ('leading', 69), ('planning', 66), ('make', 60), ('understanding', 57), ('working', 50), ('creating', 40), ('improving', 30), ('managing', 20), ('supporting', 10), ('providing', 10), ('offering', 10), ('delivering', 10), ('implementing', 10), ('executing', 10), ('operating', 10), ('maintaining', 10)]
```

```
adj_df = pd.DataFrame(adj_counter.most_common(15), columns=["adjective", "freq"])
noun_df = pd.DataFrame(noun_counter.most_common(15), columns=["noun", "freq"])

plt.figure(figsize=(9, 5))
plt.barh(adj_df["adjective"][:-1], adj_df["freq"][:-1])
plt.title("Top Adjectives in Administrative Job Descriptions")
plt.xlabel("Frequency")
plt.ylabel("Adjective")
plt.show()
```



Clustering

```
X_cluster = TfidfVectorizer(
    ngram_range=(1, 2),
    min_df=3,
    max_df=0.75
).fit_transform(df_admin["text_lemma"])
```

```
silhouette_results = {}

for k in range(2, 7):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = model.fit_predict(X_cluster)
    score = silhouette_score(X_cluster, labels)
    silhouette_results[k] = score

silhouette_results
```

```
{2: np.float64(0.04246865872420617),
 3: np.float64(0.06949566133543136),
 4: np.float64(0.07655787582232243),
 5: np.float64(0.09411823621880856),
 6: np.float64(0.10327029113706004)}
```

```
best_k = max(silhouette_results, key=silhouette_results.get)
print("k =", best_k)
```

```
k = 6
```

```
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
df_admin["cluster"] = kmeans.fit_predict(X_cluster)
```

```
df_admin["cluster"].value_counts().sort_index()
```

```

count
cluster
0      12
1      17
2      15
3     150
4     103
5      24

```

dtype: int64

```

cluster_vectorizer = TfidfVectorizer(
    ngram_range=(1,2),
    min_df=3,
    max_df=0.75
)
X_cluster_terms = cluster_vectorizer.fit_transform(df_admin["text_lemma"])
terms = np.array(cluster_vectorizer.get_feature_names_out())

order_centroids = kmeans.cluster_centers_.argsort()[:, :-1]

for i in range(best_k):
    top_terms = terms[order_centroids[i, :-1]]
    print(f"Cluster {i} top terms:")
    print(", ".join(top_terms))
    print("-" * 80)

```

Cluster 0 top terms:
accountancy, people, day, management, hold least, heathy environment, relatations, consolidation hold, relatations bank, european heathy, fol

Cluster 1 top terms:
location, full time, type, time sector, type full, customer, branch, full, sale, time, sector, safe

Cluster 2 top terms:
rota, cleaning, assist, area, holiday, scheme, floor, plus hoc, rota holiday, rota training, stairwell, staffing rota

Cluster 3 top terms:
financial, accounting, management, finance, team, business, process, office, support, und, control, laboratory

Cluster 4 top terms:
marketing, sale, product, business, customer, team, market, digital, strategy, development, brand, new

Cluster 5 top terms:
environmental, site, cleaning, equipment, appropriate, employee, procedure, management, action, regulation, environmental management, chemica

Examine the correspondence between clusters and job titles

```

cluster_job_table = pd.crosstab(df_admin["cluster"], df_admin["job"])
cluster_job_table

```

job	administrative manager	chief accountant	chief financial officer	cleaning manager	environmental manager	export sales manager	laboratory manager	marketing director	marketing manager	planning director
cluster										
0	0	12	0	0	0	0	0	0	0	0
1	0	3	0	1	0	0	0	0	0	0
2	0	0	0	15	0	0	0	0	0	0
3	14	24	36	3	3	0	14	2	5	10
4	0	0	0	0	0	11	0	16	21	1
-	-	-	-	-	-	-	-	-	-	-

Topic Modeling

```
lda_vectorizer = CountVectorizer(
    ngram_range=(1,2),
    min_df=3,
    max_df=0.75
)

X_lda = lda_vectorizer.fit_transform(df_admin["text_lemma"])
feature_names = np.array(lda_vectorizer.get_feature_names_out())
```

```
n_topics = 4
lda_model = LatentDirichletAllocation(
    n_components=n_topics,
    random_state=42,
    learning_method="batch"
)

doc_topic = lda_model.fit_transform(X_lda)
```

```
for topic_idx, topic in enumerate(lda_model.components_):
    top_terms = feature_names[topic.argsort()[-12:]][::-1]
    print(f"Topic {topic_idx + 1}:")
    print(", ".join(top_terms))
    print("-" * 80)
```

```
Topic 1:
team, marketing, business, management, product, customer, sale, client, market, strategy, development, support
-----
Topic 2:
site, cleaning, training, area, customer, service, employee, accounting, management, scheme, laboratory, audit
-----
Topic 3:
financial, sale, management, business, accounting, finance, service, team, year, tax, process, english
-----
Topic 4:
customer, time, sale, und, team, full, type, location, sector, full time, branch, finance
-----
```

```
df_admin["dominant_topic"] = doc_topic.argmax(axis=1) + 1
df_admin["dominant_topic"].value_counts().sort_index()
```

	count
dominant_topic	
1	179
2	39
3	53
4	50

dtype: int64

```
topic_job_table = pd.crosstab(df_admin["dominant_topic"], df_admin["job"])
topic_job_table
```

	job	administrative manager	chief accountant	chief financial officer	cleaning manager	environmental manager	export sales manager	laboratory manager	marketing director	marketing manager	pl
dominant_topic											
1		6	13	9	1	12	7	10	17	22	
2		0	5	0	29	1	0	3	0	0	
3		2	18	15	3	0	4	0	1	0	
4		0	0	0	0	0	0	0	0	0	

LDA visualization

```
import matplotlib.pyplot as plt
import numpy as np
import math
```

```
def plot_lda_topics(lda_model, feature_names, n_top_words=8, n_cols=2):
    n_topics = lda_model.n_components
    n_rows = math.ceil(n_topics / n_cols)

    fig, axes = plt.subplots(n_rows, n_cols, figsize=(14, 4 * n_rows))
    axes = np.array(axes).reshape(-1)

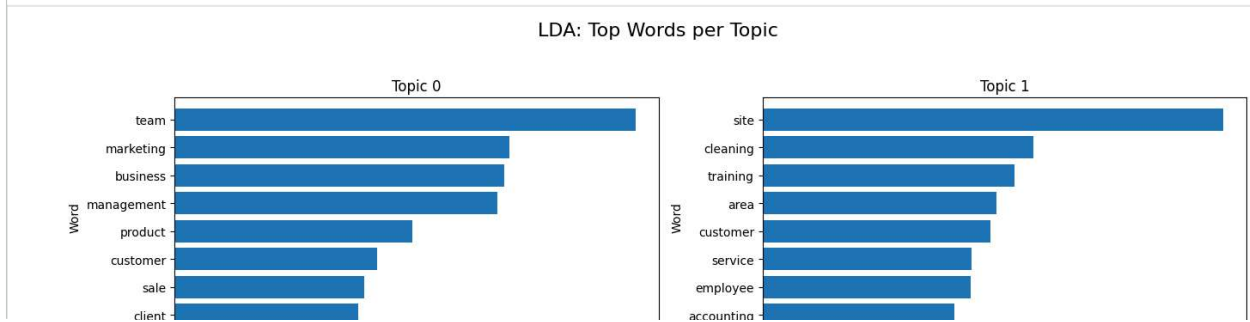
    for topic_idx, topic in enumerate(lda_model.components_):
        top_indices = topic.argsort()[::-1][:n_top_words]
        top_terms = feature_names[top_indices]
        top_weights = topic[top_indices]

        ax = axes[topic_idx]
        ax.barh(top_terms[:-1], top_weights[:-1])
        ax.set_title(f"Topic {topic_idx}")
        ax.set_xlabel("Weight")
        ax.set_ylabel("Word")

    for j in range(topic_idx + 1, len(axes)):
        fig.delaxes(axes[j])

    plt.suptitle("LDA: Top Words per Topic", fontsize=16)
    plt.tight_layout(rect=[0, 0, 1, 0.96])
```

```
plot_lda_topics(lda_model, feature_names, n_top_words=8, n_cols=2)
```



```
import pandas as pd
import numpy as np
import nltk
import re
import ast
import matplotlib.pyplot as plt

from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from nltk.corpus import stopwords

nltk.download('punkt')
nltk.download('wordnet')
nltk.download('stopwords')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
True
```

```
from google.colab import files
uploaded = files.upload()
```

选择文件 1.csv
1.csv(text/csv) – 5585802 bytes, last modified: 2026/3/14 – 100% done
Saving 1.csv to 1.csv

```
df = pd.read_csv("1.csv", encoding="latin1")
print(df.shape)
df.head()
```

(4412, 7)

	Unnamed: 0	ISCO	major_job	job	position	location	description
0	0	21	SCIENCE AND ENGINEERING PROFESSIONALS	physicist	Accelerator Physicist id54315	Villigen PSI, Aargau	[' You have an academic degree in physics or e...
1	1	21	SCIENCE AND ENGINEERING PROFESSIONALS	physicist	Applied Physicist (Computing) (EP–LBC–2021–125...	Geneva	[' Be in charge of the development of applicat...
2	2	21	SCIENCE AND ENGINEERING PROFESSIONALS	physicist	Accelerator Physicist (BE–ABP–LNO–2021–122–LD)...	Geneva	[' Contribute to the maintenance and developme...
3	3	21	SCIENCE AND ENGINEERING PROFESSIONALS	physicist	Medical Devices Physicist	Newton, Cambridgeshire	[' Agency: Newton Colmore Consulting Reference...
4	4	21	SCIENCE AND ENGINEERING PROFESSIONALS	physicist	Fluidics Physicist	Cambridge	[' Agency: Newton Colmore Consulting Reference...

后续步骤: [使用 df 生成代码](#) [New interactive sheet](#)

```
print(df.columns.tolist())
print(df["major_job"].value_counts())

['Unnamed: 0', 'ISCO', 'major_job', 'job', 'position', 'location', 'description']
major_job
SCIENCE AND ENGINEERING PROFESSIONALS      885
HEALTH PROFESSIONALS                        626
INFORMATION AND COMMUNICATIONS TECHNOLOGY PROFESSIONALS  447
ADMINISTRATIVE AND COMMERCIAL MANAGERS      321
BUSINESS AND ADMINISTRATION PROFESSIONALS    288
BUSINESS AND ADMINISTRATION ASSOCIATE PROFESSIONALS  253
TEACHING PROFESSIONALS                      252
CUSTOMER SERVICES CLERKS                   239
DRIVERS AND MOBILE PLANT OPERATORS          211
CLEANERS AND HELPERS                       202
SALES WORKERS                              185
MARKET-ORIENTED SKILLED AGRICULTURAL WORKERS  121
PROTECTIVE SERVICES WORKERS                 111
SCIENCE AND ENGINEERING ASSOCIATE PROFESSIONALS  104
INFORMATION AND COMMUNICATIONS TECHNICIANS    94
HEALTH ASSOCIATE PROFESSIONALS               73
Name: count, dtype: int64
```

```
ic_df = df[df["major_job"].str.contains("INFORMATION AND COMMUNICATION", na=False)].copy()

print(ic_df.shape)
print(ic_df["major_job"].value_counts())
```

```
(541, 7)
major_job
INFORMATION AND COMMUNICATIONS TECHNOLOGY PROFESSIONALS  447
INFORMATION AND COMMUNICATIONS TECHNICIANS              94
Name: count, dtype: int64
```

```
def parse_description(text):
    try:
        text = ast.literal_eval(text)
        if isinstance(text, list):
            return " ".join(text)
        else:
            return str(text)
    except:
        return str(text)

ic_df["text"] = ic_df["description"].fillna("").apply(parse_description)

print(ic_df["text"].iloc[0]:1000)
```

Very competitive salary 40 hours per week Pension scheme Life insurance 25 days annual leave Ensuring the Safe and efficient operation of the plant during steady

Preprocessing: lowercases the text, tokenizes it, removes stopwords, and lemmatizes the words

```
def preprocess_text(text, remove_stopwords=True, use_lemma=True):
    tokens = word_tokenize(text.lower())
    tokens = [word for word in tokens if word.isalpha()]

    if remove_stopwords:
        stop_words = set(stopwords.words('english'))
        tokens = [word for word in tokens if word not in stop_words]

    if use_lemma:
        lemmatizer = WordNetLemmatizer()
        tokens = [lemmatizer.lemmatize(word) for word in tokens]

    return tokens
```

```
ic_df["tokens"] = ic_df["text"].apply(preprocess_text)
ic_df["clean_text"] = ic_df["tokens"].apply(lambda x: " ".join(x))

ic_df[["job", "position", "clean_text"]].head()
```

	job	position	clean_text	
2998	operations technician	Shift Operations Technician	competitive salary hour per week pension schem...	
2999	operations technician	FÄ¼r Freelancer: Operations Technician in Hamburg	diagnosis troubleshoots repair customer networ...	
3000	operations technician	Operations Technician	engages early project sap regard agreed electr...	
3001	operations technician	Shift Operations Technician	starting stopping monitoring boiler steam turb...	
3002	operations technician	Operations Technician –Training Provided	substantial career growth path great overall e...	

Word Frequency Analysis

```
from collections import Counter
all_words = []

for tokens in ic_df["tokens"]:
    all_words.extend(tokens)

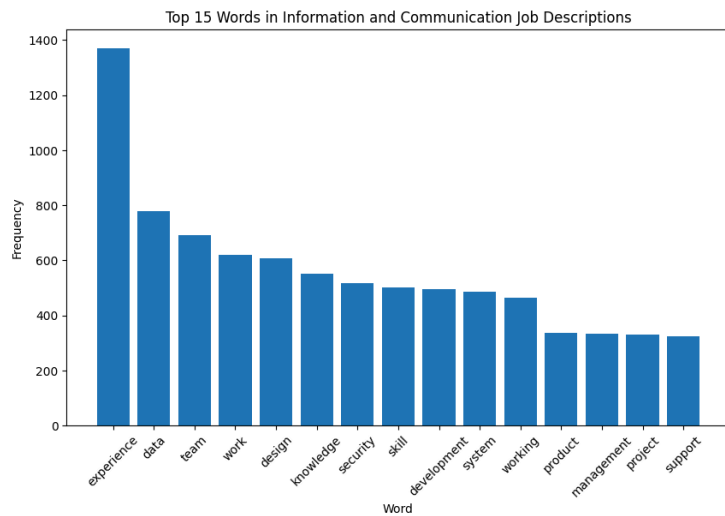
word_freq = pd.DataFrame(Counter(all_words).most_common(30), columns=["word", "freq"])
word_freq
```

	word	freq	
0	experience	1370	
1	data	777	
2	team	690	
3	work	619	
4	design	608	
5	knowledge	550	
6	security	517	
7	skill	500	
8	development	495	
9	system	487	
10	working	464	
11	product	335	
12	management	334	
13	project	330	
14	support	325	
15	understanding	309	
16	technical	302	
17	database	298	
18	software	288	
19	ability	278	
20	good	274	
21	process	273	
22	business	271	
23	solution	270	
24	tool	263	
25	technology	259	
26	application	254	
27	strong	254	
28	environment	251	
29	year	245	

后续步骤: [使用 word_freq 生成代码](#) [New interactive sheet](#)

```
top_words_plot = word_freq.head(15)

plt.figure(figsize=(10,6))
plt.bar(top_words_plot["word"], top_words_plot["freq"])
plt.xticks(rotation=45)
plt.title("Top 15 Words in Information and Communication Job Descriptions")
plt.xlabel("Word")
plt.ylabel("Frequency")
plt.show()
```



Many of the common words are related to technical work, such as data, knowledge, security, skill, development, and systems. Besides, there are some broader job-related terms, such as experience, team, and work.

Bigram Analysis

```
def make_ngrams(tokens, n):
    return [tuple(tokens[i:i+n]) for i in range(len(tokens) - n + 1)]

all_bigrams = []
for tokens in ic_df["tokens"]:
    all_bigrams.extend(make_ngrams(tokens, 2))

bigram_freq = pd.DataFrame(
    Counter(all_bigrams).most_common(30),
    columns=["bigram", "freq"]
)

bigram_freq["bigram"] = bigram_freq["bigram"].apply(lambda x: " ".join(x))
bigram_freq
```

	bigram	freq	
0	communication skill	131	
1	experience working	125	
2	machine learning	110	
3	year experience	99	
4	computer science	81	
5	data science	71	
6	best practice	61	
7	ability work	59	
8	good knowledge	56	
9	good understanding	54	
10	software development	53	
11	information security	52	
12	team member	52	
13	cyber security	46	
14	least year	43	
15	problem solving	41	
16	sql server	41	
17	social medium	38	
18	flexible working	38	
19	experience using	34	
20	competitive salary	33	
21	previous experience	33	
22	operating system	32	
23	skill ability	32	
24	attention detail	32	
25	video content	32	
26	programming language	32	
27	good communication	31	
28	work experience	31	
29	fluent english	31	

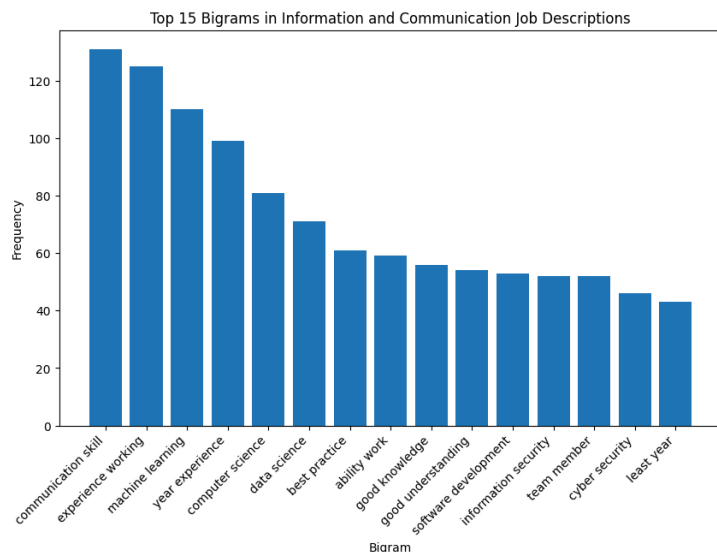
后续步骤:

[使用 bigram_freq 生成代码](#)

[New interactive sheet](#)

```
top_bigrams_plot = bigram_freq.head(15)
```

```
plt.figure(figsize=(10,6))
plt.bar(top_bigrams_plot["bigram"], top_bigrams_plot["freq"])
plt.xticks(rotation=45, ha="right")
plt.title("Top 15 Bigrams in Information and Communication Job Descriptions")
plt.xlabel("Bigram")
plt.ylabel("Frequency")
plt.show()
```



It shows the technical skills such as machine learning, computer science, data science, software development, and information security. There are also communication skill and team member, which suggests that employers are not only emphasizing technical expertise, but also interpersonal and collaborative abilities.

✓ Trait Lexicon Analysis

I created a custom trait lexicon to identify candidate traits mentioned in job descriptions. Each trait category contains a list of related keywords.

```
trait_lexicon = {
    "communication": [
        "communication", "communicate", "communicating",
        "written", "verbal", "presentation", "interpersonal"
    ],
    "teamwork": [
        "team player", "teamwork", "collaboration", "collaborate",
        "collaborative", "cross functional", "cross-functional"
    ],
    "problem_solving": [
        "problem solving", "solve", "solving", "solution",
        "troubleshooting", "analytical", "analyze", "analysis"
    ],
    "detail_orientation": [
        "attention detail", "detail oriented", "detail-oriented",
        "accuracy", "accurate"
    ],
    "leadership": [
        "leadership", "lead", "leading", "mentor", "mentoring",
        "manage", "management"
    ],
    "adaptability": [
        "adaptable", "adaptability", "flexible", "flexibility",
        "fast paced", "fast-paced", "multitask", "multi task"
    ],
    "creativity": [
        "creative", "creativity", "innovative", "innovation"
    ],
    "independence": [
        "independent", "independently", "self motivated",
        "self-motivated", "initiative", "self starter", "self-starter"
    ],
    "time_management": [
        "time management", "deadline", "deadlines", "prioritize",
        "prioritise", "organized", "organised"
    ]
}
```

```
def has_trait(text, keywords):
    text = str(text).lower()
    for word in keywords:
        if word in text:
            return 1
    return 0

for trait, keywords in trait_lexicon.items():
    ic_df[trait] = ic_df["clean_text"].apply(lambda x: has_trait(x, keywords))

trait_cols = list(trait_lexicon.keys())

trait_summary = pd.DataFrame({
    "trait": trait_cols,
    "count": [ic_df[col].sum() for col in trait_cols]
})
```

```
trait_summary["percentage"] = round(trait_summary["count"] / len(ic_df) * 100, 2)
trait_summary = trait_summary.sort_values(by="count", ascending=False)
```

```
trait_summary
```

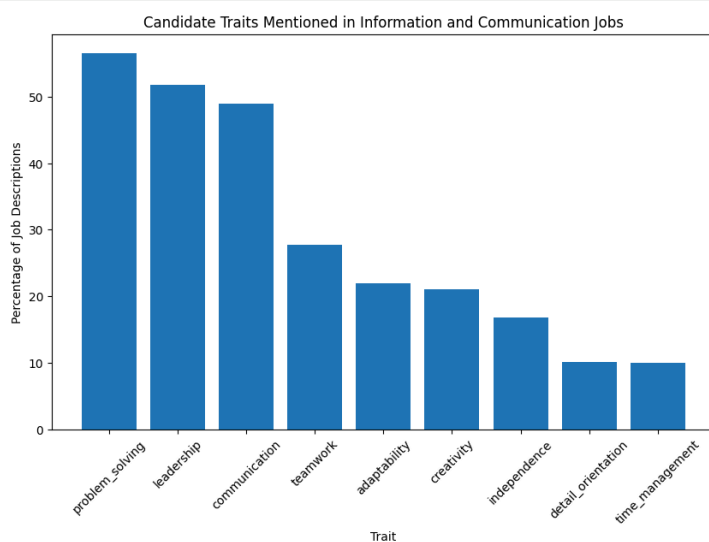
	trait	count	percentage
2	problem_solving	306	56.56
4	leadership	280	51.76
0	communication	265	48.98
1	teamwork	150	27.73
5	adaptability	119	22.00
6	creativity	114	21.07
7	independence	91	16.82
3	detail_orientation	55	10.17
8	time_management	54	9.98

后续步骤:

[使用 trait_summary 生成代码](#)

[New interactive sheet](#)

```
plt.figure(figsize=(10,6))
plt.bar(trait_summary["trait"], trait_summary["percentage"])
plt.xticks(rotation=45)
plt.title("Candidate Traits Mentioned in Information and Communication Jobs")
plt.xlabel("Trait")
plt.ylabel("Percentage of Job Descriptions")
plt.show()
```



The most frequently mentioned traits are problem-solving, leadership, communication, and teamwork. This suggests that firms in this category value candidates who can solve problems, communicate clearly, and work effectively with others.

TF-IDF Analysis

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vec = TfidfVectorizer()
X_tfidf = tfidf_vec.fit_transform(ic_df["clean_text"])

tfidf_df = pd.DataFrame(
    X_tfidf.toarray(),
    columns=tfidf_vec.get_feature_names_out()
)

print(tfidf_df.shape)

(541, 5676)
```

```
mean_tfidf = tfidf_df.mean(axis=0).sort_values(ascending=False)

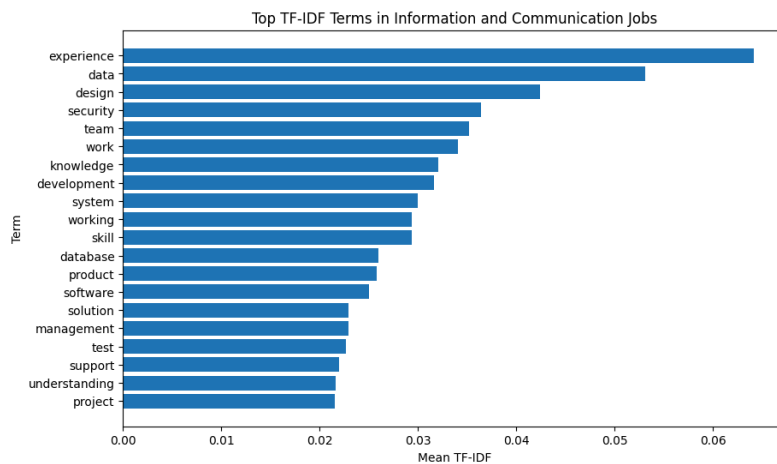
top_tfidf_terms = pd.DataFrame({
    "term": mean_tfidf.index[:20],
    "mean_tfidf": mean_tfidf.values[:20]
})

top_tfidf_terms
```

	term	mean_tfidf
0	experience	0.064162
1	data	0.053061
2	design	0.042368
3	security	0.036410
4	team	0.035180
5	work	0.034072
6	knowledge	0.032073
7	development	0.031590
8	system	0.029962
9	working	0.029398
10	skill	0.029340
11	database	0.025957
12	product	0.025816
13	software	0.025050
14	solution	0.022917
15	management	0.022901
16	test	0.022699
17	support	0.021975
18	understanding	0.021583
19	project	0.021553

后续步骤: [使用 top_tfidf_terms 生成代码](#) [New interactive sheet](#)

```
plt.figure(figsize=(10,6))
plt.barh(top_tfidf_terms["term"][:-1], top_tfidf_terms["mean_tfidf"][:-1])
plt.title("Top TF-IDF Terms in Information and Communication Jobs")
plt.xlabel("Mean TF-IDF")
plt.ylabel("Term")
plt.show()
```



The figure shows a strong emphasis on technical areas such as data, security, knowledge, development, system, skill, databases, and software.

Conclusion

The results show that information and communication job descriptions emphasize both technical competencies and soft skills. The most frequently mentioned traits are problem-solving, leadership, communication, and teamwork. In addition, word frequency, bigram, and TF-IDF analyses show that this category strongly focuses on data, software, systems, databases, and security-related knowledge.

GR5067: Final Group Project

```
In [53]: import numpy as np
import pandas as pd
import re
import ast
import matplotlib.pyplot as plt
import seaborn as sns
import spacy

from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.model_selection import train_test_split, cross_val_score, Strat
from sklearn.pipeline import Pipeline
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion

from IPython.display import display
```

```
In [2]: # load the dataset
df = pd.read_csv("JD_data.csv")

# inspect the dataset
print(df.shape)
print(df.columns.tolist())

print(df.head())
```

```
(4412, 7)
['Unnamed: 0', 'ISCO', 'major_job', 'job', 'position', 'location', 'description']
  Unnamed: 0  ISCO          major_job      job \
0          0    21  SCIENCE AND ENGINEERING PROFESSIONALS  physicist
1          1    21  SCIENCE AND ENGINEERING PROFESSIONALS  physicist
2          2    21  SCIENCE AND ENGINEERING PROFESSIONALS  physicist
3          3    21  SCIENCE AND ENGINEERING PROFESSIONALS  physicist
4          4    21  SCIENCE AND ENGINEERING PROFESSIONALS  physicist

          position          location
\
0          Accelerator Physicist id54315  Villigen PSI, Aargau
1  Applied Physicist (Computing) (EP-LBC-2021-125...  Geneva
2  Accelerator Physicist (BE-ABP-LN0-2021-122-LD)...  Geneva
3          Medical Devices Physicist  Newton, Cambridgeshire
4          Fluidics Physicist  Cambridge

          description
0  [' You have an academic degree in physics or e...
1  [' Be in charge of the development of applicat...
2  [' Contribute to the maintenance and developme...
3  [' Agency: Newton Colmore Consulting Reference...
4  [' Agency: Newton Colmore Consulting Reference...
```

```
In [3]: print(df.isna().sum().sort_values(ascending=False).head(20))
```

```
Unnamed: 0      0
ISCO           0
major_job       0
job             0
position        0
location        0
description     0
dtype: int64
```

```
In [4]: print(df["major_job"].value_counts().head(10))
```

```
major_job
SCIENCE AND ENGINEERING PROFESSIONALS      885
HEALTH PROFESSIONALS                       626
INFORMATION AND COMMUNICATIONS TECHNOLOGY PROFESSIONALS  447
ADMINISTRATIVE AND COMMERCIAL MANAGERS      321
BUSINESS AND ADMINISTRATION PROFESSIONALS    288
BUSINESS AND ADMINISTRATION ASSOCIATE PROFESSIONALS  253
TEACHING PROFESSIONALS                     252
CUSTOMER SERVICES CLERKS                   239
DRIVERS AND MOBILE PLANT OPERATORS          211
CLEANERS AND HELPERS                        202
Name: count, dtype: int64
```

```

In [5]: top_major = ["INFORMATION AND COMMUNICATIONS TECHNOLOGY PROFESSIONALS", "SCI
df2 = df[df["major_job"].isin(top_major)].copy()

In [6]: # In this dataset, the description column is stored like a string representa
def parse_desc(x):
    try:
        val = ast.literal_eval(x)
        if isinstance(val, list):
            return " ".join(str(v) for v in val)
        return str(val)
    except Exception:
        return str(x)

df2["desc_text"] = df2["description"].apply(parse_desc)

In [7]: import spacy
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS

# Load spaCy model
try:
    nlp = spacy.load("en_core_web_sm")
except OSError:
    import subprocess
    import sys
    subprocess.check_call([sys.executable, "-m", "spacy", "download", "en_co
    nlp = spacy.load("en_core_web_sm")

# Keep a few words
custom_stop = set(ENGLISH_STOP_WORDS) - {"not", "no", "never"}

In [12]: # Text cleaning + bag-of-words preprocessing
# lowercase, tokenize with spaCy, remove stopwords/punctuation

def process_to_bag(text):
    text = str(text).lower()
    text = re.sub(r"http\S+|www\S+", " ", text)
    text = re.sub(r"\S+@\S+", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    # remove most non-letter symbols, but keep + and # for things like C++ /
    text = re.sub(r"^[a-zA-Z0-9+# ]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()

    doc = nlp(text)
    bag = []

    for token in doc:
        lemma = token.lemma_.lower().strip()

        # skip empty strings

```

```
    if not lemma:
        continue

    # keep some short technical words manually
    if lemma in {"r", "sql", "bi", "ml", "ai"}:
        bag.append(lemma)
        continue

    # remove stopwords / punctuation / spaces
    if token.is_stop or token.is_punct or token.is_space:
        continue

    if lemma in custom_stop:
        continue

    # keep regular alphabetic words only
    if token.is_alpha:
        bag.append(lemma)

    return bag

df2["tokens"] = df2["desc_text"].apply(process_to_bag)
df2["text_clean"] = df2["tokens"].apply(lambda x: " ".join(x))
df2["word_count"] = df2["tokens"].apply(len)
```

```
In [15]: # My focus major job category
sub = df2[df2["major_job"] == "SCIENCE AND ENGINEERING PROFESSIONALS"].copy()
print(sub["job"].value_counts())
```

job	
physicist	49
chemist	48
process chemist	48
statistician	45
ecologist	44
physiologist	44
geologist	42
biostatistician	42
biologist	40
inspection engineer	36
microbiologist	35
biochemist	32
mechanical designer	31
maintenance engineer	28
project engineer	26
verification engineer	26
process engineer	23
hydrogeologist	22
mathematician	20
environmental engineer	20
toxicologist	19
construction planner	19
development engineer	18
product developer	16
molecular biologist	15
mechatronics engineer	15
mechanical engineer	14
plant engineer	13
organic chemist	13
testing engineer	13
instrument engineer	12
geneticist	12
immunologist	5

Name: count, dtype: int64

```
In [16]: # Most frequent words and phrases
# Here I look at the most common unigrams and bigrams across all jobs, and t

from sklearn.feature_extraction.text import CountVectorizer

def get_top_ngrams(text_series, ngram_range=(1,1), top_n=15):
    vec = CountVectorizer(ngram_range=ngram_range, min_df=2)
    X = vec.fit_transform(text_series)

    counts = X.sum(axis=0).A1
    terms = vec.get_feature_names_out()

    df_ng = pd.DataFrame({
        "term": terms,
```

```

        "count": counts
    }).sort_values("count", ascending=False)

    return df_ng.head(top_n)

# ---- Overall ----
top_uni_overall = get_top_ngrams(sub["text_clean"], (1,1), 15)
top_bi_overall = get_top_ngrams(sub["text_clean"], (2,2), 15)

print("\nTop unigrams (overall):")
print(top_uni_overall)

print("\nTop bigrams (overall):")
print(top_bi_overall)

```

Top unigrams (overall):

	term	count
1266	experience	1940
3787	work	1860
2751	project	1135
3437	team	915
951	development	850
2719	process	847
933	design	792
1910	knowledge	789
3193	skill	786
3394	support	689
3444	technical	630
2073	management	611
144	analysis	609
869	datum	579
3	ability	567

Top bigrams (overall):

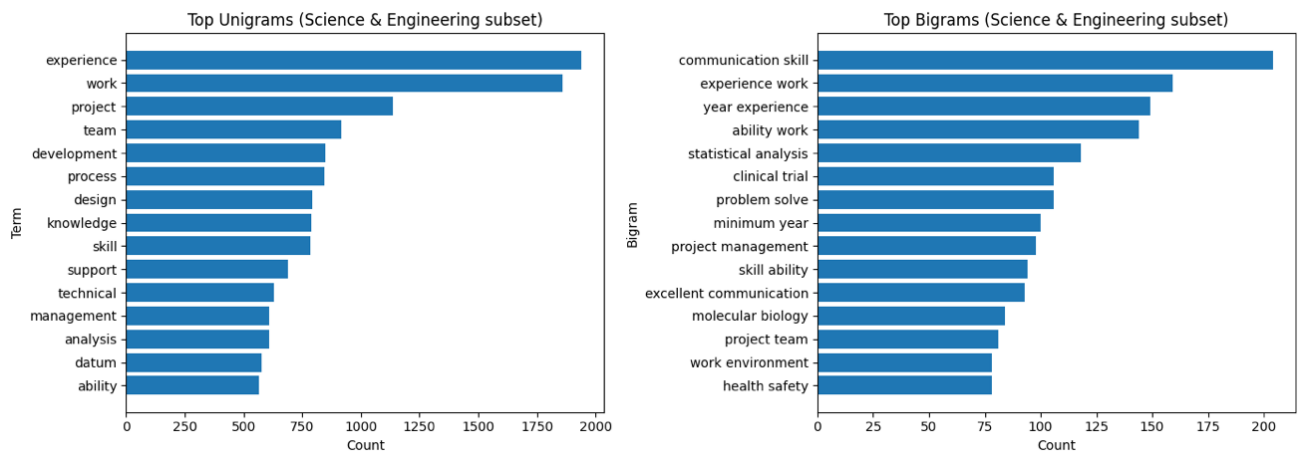
	term	count
2538	communication skill	204
5946	experience work	159
17763	year experience	149
77	ability work	144
15256	statistical analysis	118
2281	clinical trial	106
11784	problem solve	106
9873	minimum year	100
12334	project management	98
14655	skill ability	94
5526	excellent communication	93
9964	molecular biology	84
12400	project team	81
17491	work environment	78
7019	health safety	78

```
In [17]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].barh(top_uni_overall["term"][::-1], top_uni_overall["count"][::-1])
axes[0].set_title("Top Unigrams (Science & Engineering subset)")
axes[0].set_xlabel("Count")
axes[0].set_ylabel("Term")

axes[1].barh(top_bi_overall["term"][::-1], top_bi_overall["count"][::-1])
axes[1].set_title("Top Bigrams (Science & Engineering subset)")
axes[1].set_xlabel("Count")
axes[1].set_ylabel("Bigram")

plt.tight_layout()
plt.show()
```



```
In [61]: # Word frequency by job
jobs = sub["job"].unique()[:2]

for j in jobs:
    print(f"{j.upper()}")
    subset = sub[sub["job"] == j]

    print("\nTop unigrams:")
    print(get_top_ngrams(subset["text_clean"], (1,1), 10))

    print("\nTop bigrams:")
    print(get_top_ngrams(subset["text_clean"], (2,2), 10))
```

PHYSICIST

Top unigrams:

	term	count
685	work	141
221	experience	111
162	development	96
323	knowledge	86
158	design	71

630	team	71
463	physics	64
402	new	64
273	high	58
52	beam	56

Top bigrams:

	term	count
536	knowledge application	32
911	require need	27
656	need organization	27
1040	team goal	20
249	design development	19
672	night sunday	17
457	high level	17
959	share knowledge	17
1021	sunday official	17
683	official holiday	17

BIOCHEMIST

Top unigrams:

	term	count
214	experience	79
603	work	77
463	protein	72
63	biochemistry	41
564	team	36
530	skill	32
67	biology	32
373	molecular	31
461	project	31
321	knowledge	26

Top bigrams:

	term	count
587	molecular biology	23
577	membrane protein	15
190	communication skill	12
750	protein purification	10
569	mass spectrometry	8
760	prove experience	8
344	experience membrane	8
1008	work closely	7
3	ability work	7
354	experience work	7

In [20]: `# TF-IDF`

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```

vec = TfidfVectorizer(min_df=5, max_df = 0.8, ngram_range=(1,2))
X = vec.fit_transform(sub["text_clean"])
terms = vec.get_feature_names_out()

tfidf_df = pd.DataFrame(X.toarray(), columns=terms)
tfidf_df["job"] = sub["job"].values

# Overall TF-IDF importance (whole group)
overall_tfidf = tfidf_df.drop(columns=["job"]).mean()
top_overall = overall_tfidf.sort_values(ascending=False).head(20)

print("OVERALL TF-IDF (ALL JOBS)")
print(top_overall)

```

OVERALL TF-IDF (ALL JOBS)

experience	0.046516
work	0.043997
project	0.032479
process	0.028830
design	0.026831
skill	0.025552
development	0.025234
knowledge	0.025165
team	0.024747
statistical	0.022993
technical	0.022268
analysis	0.022054
support	0.021513
datum	0.020894
product	0.020779
engineering	0.020227
management	0.019820
ability	0.019446
include	0.019223
ensure	0.019207

dtype: float64

```

In [62]: # Top TF-IDF terms by job
job_tfidf = tfidf_df.groupby("job").mean()

for job_name in job_tfidf.index[:2]:
    print(f"TOP TF-IDF TERMS: {job_name}")
    print(job_tfidf.loc[job_name].sort_values(ascending=False).head(12))

```

TOP TF-IDF TERMS: biochemist

protein	0.116001
biochemistry	0.098956
molecular	0.057836
work	0.054801
biology	0.050646
experience	0.048717
purification	0.045653
molecular biology	0.044752
characterization	0.036514
cell	0.035304
chemistry	0.035290
mass spectrometry	0.034977

Name: biochemist, dtype: float64

TOP TF-IDF TERMS: biologist

biology	0.094822
cell	0.067327
molecular	0.063135
experience	0.061022
computational	0.057768
datum	0.054340
protein	0.054047
molecular biology	0.049347
work	0.041240
november	0.040328
rna	0.039507
skill	0.039492

Name: biologist, dtype: float64

```
In [24]: import seaborn as sns
import math

plot_df = (
    job_tfidf
    .reset_index()
    .melt(id_vars="job", var_name="term", value_name="tfidf")
)

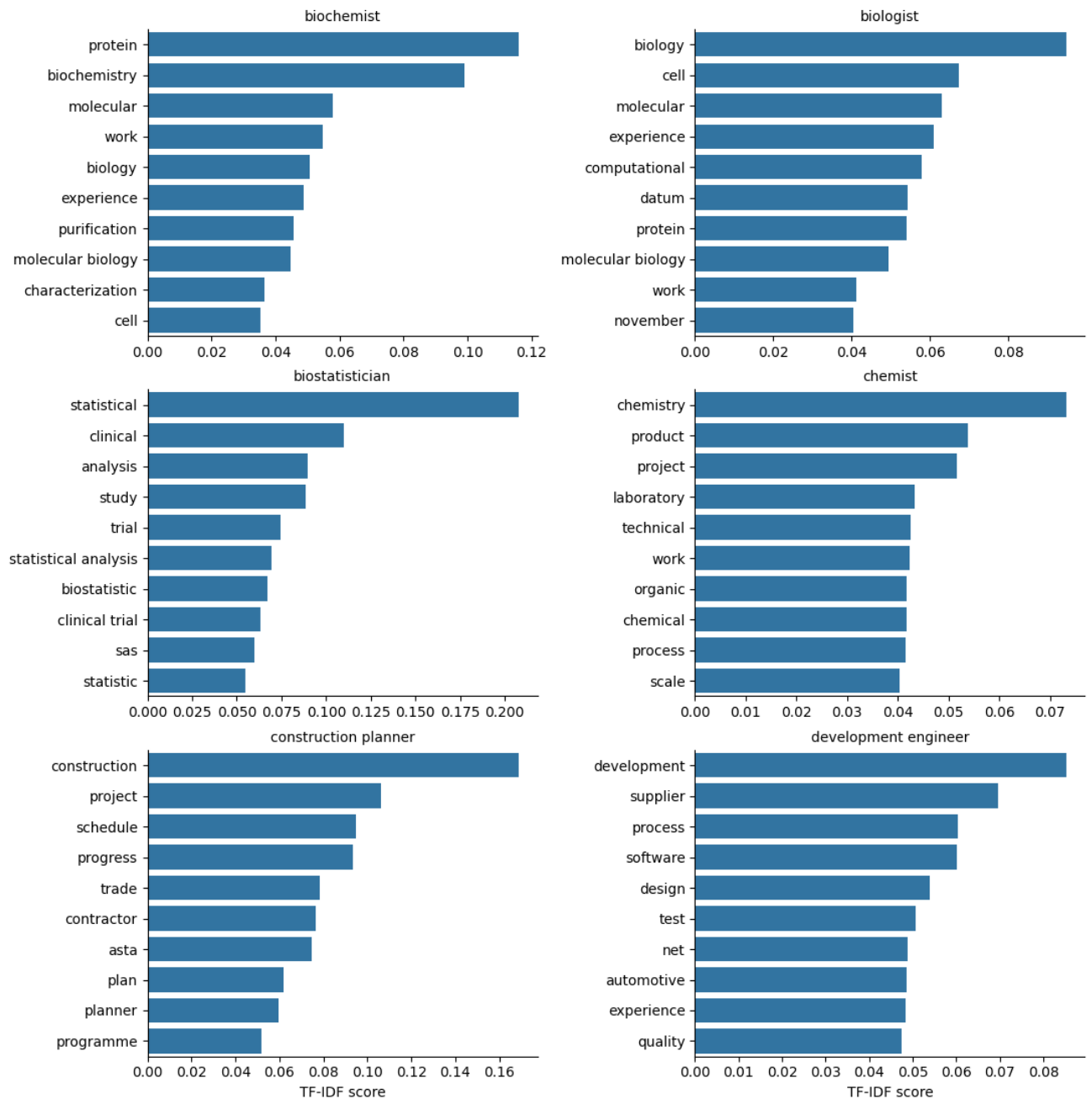
plot_top = (
    plot_df
    .sort_values(["job", "tfidf"], ascending=[True, False])
    .groupby("job")
    .head(10)
    .reset_index(drop=True)
)

selected_jobs = plot_top["job"].unique()[:6]
plot_top_small = plot_top[plot_top["job"].isin(selected_jobs)]

g = sns.FacetGrid(
```

```
    plot_top_small,  
    col="job",  
    col_wrap=2,  
    sharey=False,  
    sharex=False,  
    height=4,  
    aspect=1.4  
)  
  
g.map_dataframe(  
    sns.barplot,  
    x="tfidf",  
    y="term"  
)  
  
g.set_axis_labels("TF-IDF score", "")  
g.set_titles("{col_name}")  
  
plt.subplots_adjust(top=0.9)  
g.fig.suptitle("Top TF-IDF Terms by Job", fontsize=16, fontweight="bold")  
  
plt.show()
```

Top TF-IDF Terms by Job

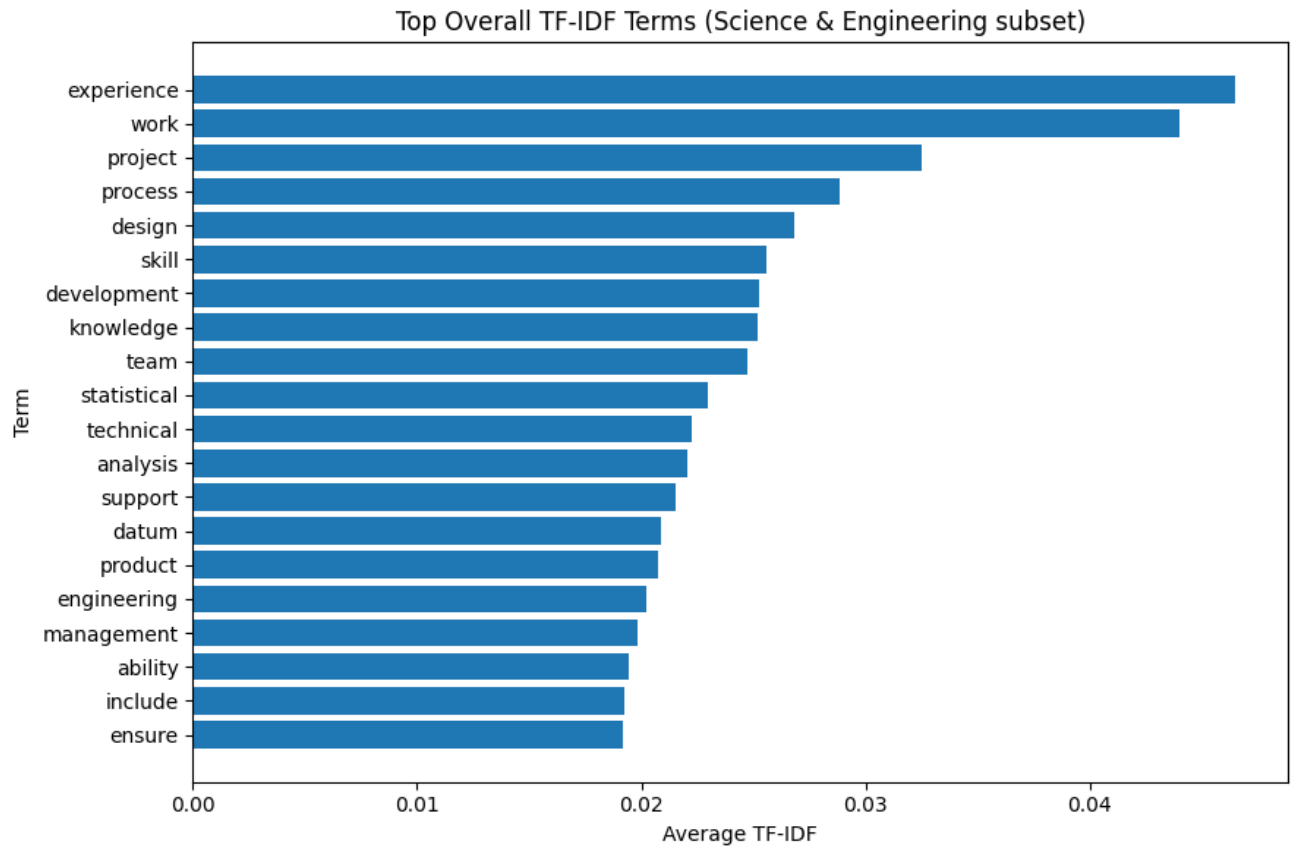


In [25]: *# Visualization for overall TF-IDF terms*

```
top_overall_df = top_overall.reset_index()
top_overall_df.columns = ["term", "tfidf_score"]

plt.figure(figsize=(9, 6))
plt.barh(top_overall_df["term"][:-1], top_overall_df["tfidf_score"][:-1])
plt.title("Top Overall TF-IDF Terms (Science & Engineering subset)")
plt.xlabel("Average TF-IDF")
plt.ylabel("Term")
```

```
plt.tight_layout()
plt.show()
```



```
In [26]: # Classification
# Here I test whether job roles can be predicted from the text alone.
```

```
X_train, X_test, y_train, y_test = train_test_split(
    sub["text_clean"],
    sub["job"],
    test_size=0.2,
    random_state=42,
    stratify=sub["job"]
)

# Cross-validation

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

print("Train size:", len(X_train))
print("Test size:", len(X_test))
```

Train size: 708

Test size: 177

```
In [29]: # Naive Bayes baseline
```

```

nb_pipe = Pipeline([
    ("vect", CountVectorizer(min_df=2, ngram_range=(1,2))),
    ("clf", MultinomialNB())
])

# Cross-validation for the original Naive Bayes model
nb_cv_scores = cross_val_score(
    nb_pipe,
    sub["text_clean"],
    sub["job"],
    cv=cv,
    scoring="f1_macro"
)

nb_pipe.fit(X_train, y_train)
nb_pred = nb_pipe.predict(X_test)

print("NAIVE BAYES RESULTS")
print("Accuracy:", round(accuracy_score(y_test, nb_pred), 3))
print("CV F1-macro scores:", np.round(nb_cv_scores, 3))
print("Mean CV F1-macro:", round(nb_cv_scores.mean(), 3))
print("Std CV F1-macro:", round(nb_cv_scores.std(), 3))

```

NAIVE BAYES RESULTS

Accuracy: 0.655

CV F1-macro scores: [0.528 0.598 0.489 0.572 0.554]

Mean CV F1-macro: 0.548

Std CV F1-macro: 0.037

```

In [56]: #Classification report
report = classification_report(y_test, nb_pred, output_dict=True)
print(pd.DataFrame(report).transpose().head(10))

```

	precision	recall	f1-score	support
biochemist	1.000000	0.500	0.666667	6.0
biologist	0.583333	0.875	0.700000	8.0
biostatistician	0.666667	0.750	0.705882	8.0
chemist	0.272727	0.600	0.375000	10.0
construction planner	1.000000	1.000	1.000000	4.0
development engineer	0.000000	0.000	0.000000	4.0
ecologist	0.750000	1.000	0.857143	9.0
environmental engineer	0.000000	0.000	0.000000	4.0
geneticist	0.000000	0.000	0.000000	2.0
geologist	0.538462	0.875	0.666667	8.0

```

/Users/yanxuejing/.pyenv/versions/3.12.6/lib/python3.12/site-packages/sklearn/metrics/_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/Users/yanxuejing/.pyenv/versions/3.12.6/lib/python3.12/site-packages/sklearn/metrics/_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/Users/yanxuejing/.pyenv/versions/3.12.6/lib/python3.12/site-packages/sklearn/metrics/_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])

```

```

In [31]: # Logistic Regression
# I also try Logistic Regression with TF-IDF

lr_pipe = Pipeline([
    ("tfidf", TfidfVectorizer(min_df=2, ngram_range=(1,2))),
    ("clf", LogisticRegression(max_iter=3000))
])

# Cross-validation for Logistic Regression
lr_cv_scores = cross_val_score(
    lr_pipe,
    sub["text_clean"],
    sub["job"],
    cv=cv,
    scoring="f1_macro"
)

lr_pipe.fit(X_train, y_train)
lr_pred = lr_pipe.predict(X_test)

print("LOGISTIC REGRESSION RESULTS")
print("Accuracy:", round(accuracy_score(y_test, lr_pred), 3))
print("CV F1-macro scores:", np.round(lr_cv_scores, 3))
print("Mean CV F1-macro:", round(lr_cv_scores.mean(), 3))
print("Std CV F1-macro:", round(lr_cv_scores.std(), 3))

```

```

LOGISTIC REGRESSION RESULTS
Accuracy: 0.672
CV F1-macro scores: [0.52  0.492 0.522 0.475 0.49 ]
Mean CV F1-macro: 0.5
Std CV F1-macro: 0.018

```

```

In [57]: #Classification report
report = classification_report(y_test, lr_pred, output_dict=True)

```

```
print(pd.DataFrame(report).transpose().head(10))
```

	precision	recall	f1-score	support
biochemist	0.750000	0.500	0.600000	6.0
biologist	0.500000	0.875	0.636364	8.0
biostatistician	0.750000	0.750	0.750000	8.0
chemist	0.370370	1.000	0.540541	10.0
construction planner	1.000000	1.000	1.000000	4.0
development engineer	0.000000	0.000	0.000000	4.0
ecologist	0.900000	1.000	0.947368	9.0
environmental engineer	0.000000	0.000	0.000000	4.0
geneticist	0.000000	0.000	0.000000	2.0
geologist	0.411765	0.875	0.560000	8.0

```
/Users/yanxuejing/.pyenv/versions/3.12.6/lib/python3.12/site-packages/sklearn/metrics/_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/Users/yanxuejing/.pyenv/versions/3.12.6/lib/python3.12/site-packages/sklearn/metrics/_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/Users/yanxuejing/.pyenv/versions/3.12.6/lib/python3.12/site-packages/sklearn/metrics/_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

In [34]: *# model comparison table*

```
comparison_df = pd.DataFrame({
    "Model": ["Naive Bayes", "Logistic Regression"],
    "Test Accuracy": [
        accuracy_score(y_test, nb_pred),
        accuracy_score(y_test, lr_pred)
    ],
    "Mean CV F1-macro": [
        nb_cv_scores.mean(),
        lr_cv_scores.mean()
    ],
    "CV Std": [
        nb_cv_scores.std(),
        lr_cv_scores.std()
    ]
}).round(3)

comparison_df
```

Out [34]:

	Model	Test Accuracy	Mean CV F1-macro	CV Std
0	Naive Bayes	0.655	0.548	0.037
1	Logistic Regression	0.672	0.500	0.018

```
In [32]: # Naive Bayes model across the four major_job categories in df2

X_train_mj, X_test_mj, y_train_mj, y_test_mj = train_test_split(
    df2["text_clean"],
    df2["major_job"],
    test_size=0.2,
    random_state=42,
    stratify=df2["major_job"]
)

nb_major_pipe = Pipeline([
    ("vect", CountVectorizer(min_df=2, ngram_range=(1,2))),
    ("clf", MultinomialNB())
])

nb_major_cv_scores = cross_val_score(
    nb_major_pipe,
    df2["text_clean"],
    df2["major_job"],
    cv=cv,
    scoring="f1_macro"
)

nb_major_pipe.fit(X_train_mj, y_train_mj)
nb_major_pred = nb_major_pipe.predict(X_test_mj)

print("NAIVE BAYES RESULTS – FOUR major_job CATEGORIES")
print("Accuracy:", round(accuracy_score(y_test_mj, nb_major_pred), 3))
print("CV F1-macro scores:", np.round(nb_major_cv_scores, 3))
print("Mean CV F1-macro:", round(nb_major_cv_scores.mean(), 3))
print("Std CV F1-macro:", round(nb_major_cv_scores.std(), 3))
```

```
NAIVE BAYES RESULTS – FOUR major_job CATEGORIES
Accuracy: 0.887
CV F1-macro scores: [0.844 0.864 0.865 0.815 0.846]
Mean CV F1-macro: 0.847
Std CV F1-macro: 0.018
```

```
In [37]: # classification report
report_major = classification_report(y_test_mj, nb_major_pred, output_dict=True)
report_major_df = pd.DataFrame(report_major).T.round(3)

summary_rows = ["accuracy", "macro avg", "weighted avg"]
```

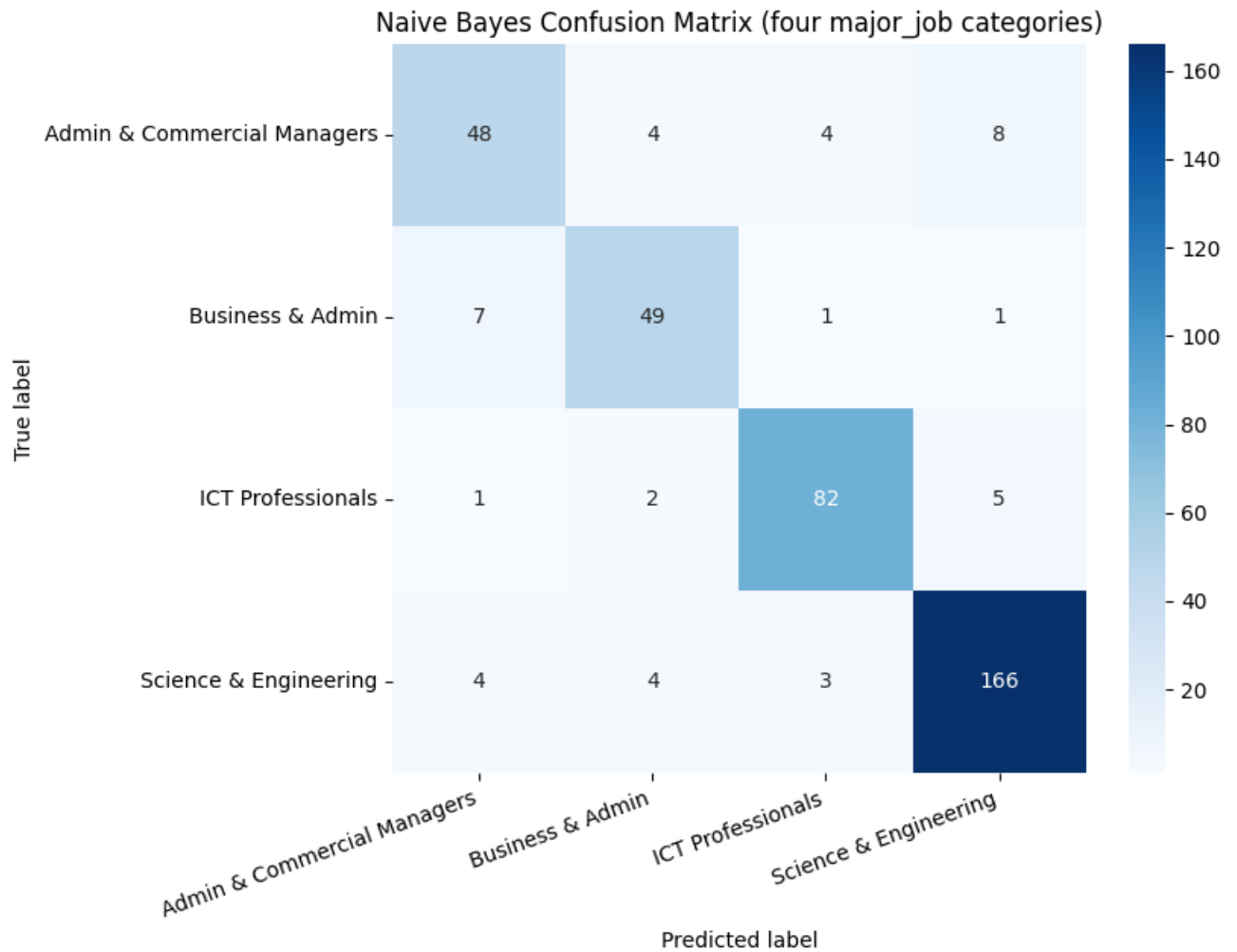
```
class_rows = [idx for idx in report_major_df.index if idx not in summary_row]
display(report_major_df.loc[class_rows + summary_rows])
```

	precision	recall	f1-score	support
ADMINISTRATIVE AND COMMERCIAL MANAGERS	0.800	0.750	0.774	64.000
BUSINESS AND ADMINISTRATION PROFESSIONALS	0.831	0.845	0.838	58.000
INFORMATION AND COMMUNICATIONS TECHNOLOGY PROFESSIONALS	0.911	0.911	0.911	90.000
SCIENCE AND ENGINEERING PROFESSIONALS	0.922	0.938	0.930	177.000
accuracy	0.887	0.887	0.887	0.887
macro avg	0.866	0.861	0.863	389.000
weighted avg	0.886	0.887	0.886	389.000

```
In [21]: # Confusion matrix for the four-category Naive Bayes model
label_short = {
    "ADMINISTRATIVE AND COMMERCIAL MANAGERS": "Admin & Commercial Managers",
    "BUSINESS AND ADMINISTRATION PROFESSIONALS": "Business & Admin",
    "INFORMATION AND COMMUNICATIONS TECHNOLOGY PROFESSIONALS": "ICT Professionals",
    "SCIENCE AND ENGINEERING PROFESSIONALS": "Science & Engineering"
}

major_classes = list(nb_major_pipe.classes_)
cm_major = confusion_matrix(y_test_mj, nb_major_pred, labels=major_classes)
cm_major_df = pd.DataFrame(
    cm_major,
    index=[label_short.get(x, x) for x in major_classes],
    columns=[label_short.get(x, x) for x in major_classes]
)

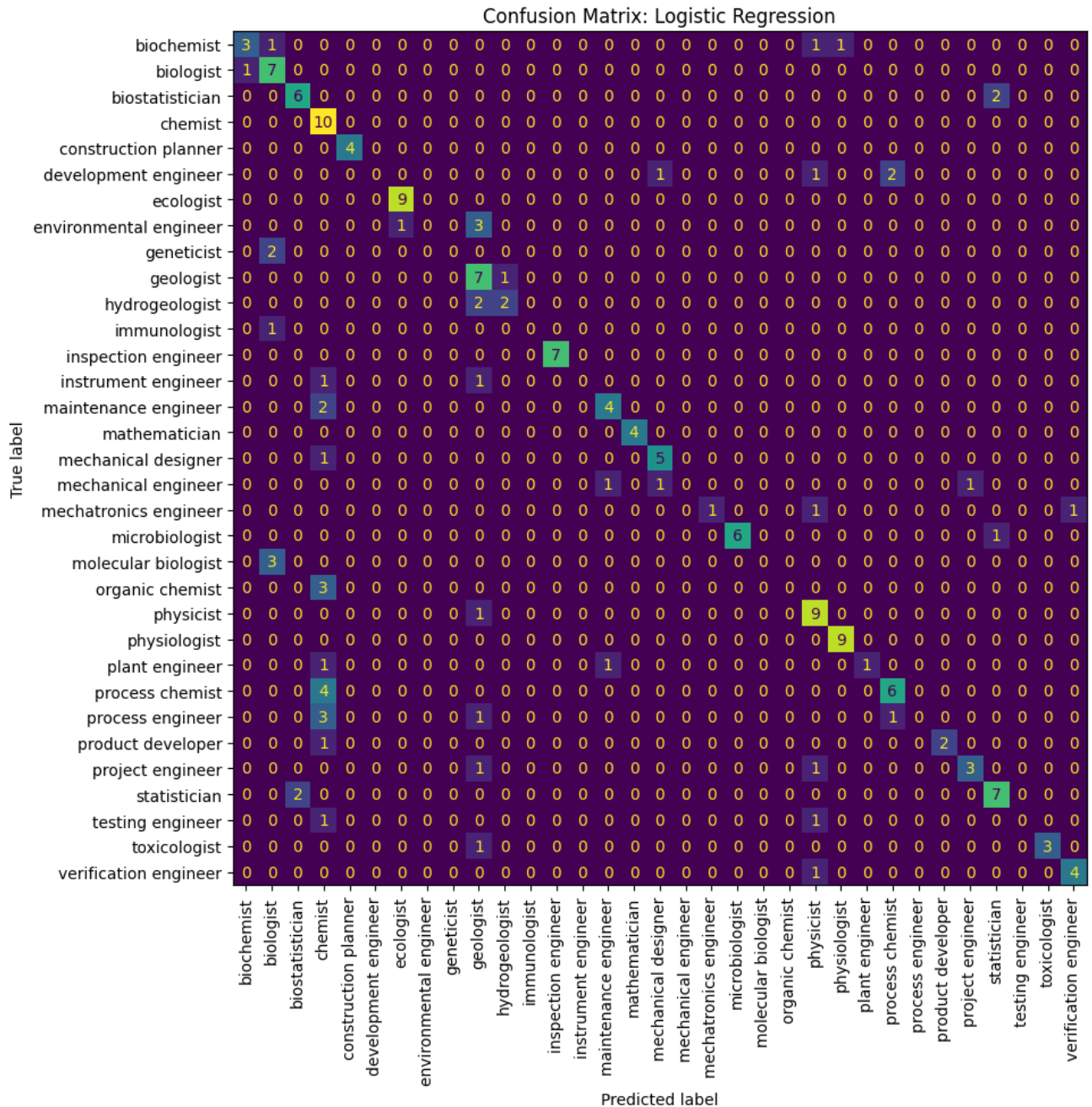
plt.figure(figsize=(8.5, 6.5))
sns.heatmap(cm_major_df, annot=True, fmt="d", cmap="Blues", cbar=True)
plt.title("Naive Bayes Confusion Matrix (four major_job categories)")
plt.xlabel("Predicted label")
plt.ylabel("True label")
plt.xticks(rotation=20, ha="right")
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```



```
In [38]: # Confusion matrix

cm = confusion_matrix(y_test, lr_pred, labels=lr_pipe.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=lr_pipe.cl

fig, ax = plt.subplots(figsize=(10, 10))
disp.plot(ax=ax, xticks_rotation=90, colorbar=False)
plt.title("Confusion Matrix: Logistic Regression")
plt.tight_layout()
plt.show()
```



In [58]: *# Top predictive words from Logistic Regression*

```
tfidf = lr_pipe.named_steps["tfidf"]
clf = lr_pipe.named_steps["clf"]

feature_names = np.array(tfidf.get_feature_names_out())

for class_idx, class_label in enumerate(clf.classes_[1:]):
    coefs = clf.coef_[class_idx]
    top_idx = coefs.argsort()[::-1][1:15]

    coef_df = pd.DataFrame({
```

```

        "term": feature_names[top_idx],
        "coefficient": coefs[top_idx]
    })

    print(f"TOP PREDICTIVE TERMS FOR {class_label}")
    print(coef_df)

```

TOP PREDICTIVE TERMS FOR biochemist

	term	coefficient
0	biochemistry	1.523736
1	protein	1.331041
2	molecular	0.694294
3	purification	0.630580
4	biology	0.550751
5	molecular biology	0.537409
6	spectrometry	0.536147
7	mass spectrometry	0.536147
8	characterization	0.525367
9	mass	0.513512
10	oligonucleotide	0.499876
11	phd	0.386777
12	prove	0.383478
13	biochemistry molecular	0.379599
14	prove experience	0.379241

TOP PREDICTIVE TERMS FOR biologist

	term	coefficient
0	biology	1.543399
1	cell	1.020301
2	datum	0.910409
3	protein	0.862672
4	ng	0.830770
5	molecular	0.816357
6	computational	0.809062
7	molecular biology	0.773867
8	ng datum	0.724330
9	november	0.685216
10	bioinformatics	0.611796
11	agency ck	0.600178
12	ck group	0.600178
13	ck	0.600178
14	group contact	0.600178

In [41]: *# LDA Topic Modeling*

```

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation
from gensim import corpora

texts = df2["tokens"].tolist()
dictionary = corpora.Dictionary(texts)

```

```
bow_corpus = [dictionary.doc2bow(doc) for doc in texts]
```

Requirement already satisfied: gensim in /Users/yanxuejing/.pyenv/versions/3.12.6/lib/python3.12/site-packages (4.4.0)
 Requirement already satisfied: numpy>=1.18.5 in /Users/yanxuejing/.pyenv/versions/3.12.6/lib/python3.12/site-packages (from gensim) (2.3.5)
 Requirement already satisfied: scipy>=1.7.0 in /Users/yanxuejing/.pyenv/versions/3.12.6/lib/python3.12/site-packages (from gensim) (1.17.1)
 Requirement already satisfied: smart_open>=1.8.1 in /Users/yanxuejing/.pyenv/versions/3.12.6/lib/python3.12/site-packages (from gensim) (7.5.1)
 Requirement already satisfied: wrapt in /Users/yanxuejing/.pyenv/versions/3.12.6/lib/python3.12/site-packages (from smart_open>=1.8.1->gensim) (2.1.2)

[notice] A new release of pip is available: 24.2 -> 26.1

[notice] To update, run: pip install --upgrade pip

```
In [42]: # Train LDA model

from gensim.models import LdaModel

K = df2["major_job"].nunique()

lda = LdaModel(
    corpus=bow_corpus,
    id2word=dictionary,
    num_topics=K,
    random_state=42,
    passes=10
)
```

```
In [55]: # Show top words per topic

print("Top words per topic:")
for t in range(K):
    top = lda.show_topic(t, topn=10)
    print(f"Topic {t}: " + " ".join(f"{w}({p:.3f})" for w, p in top))
```

Top words per topic:

Topic 0: experience(0.016) work(0.016) process(0.011) management(0.010) project(0.010) ensure(0.009) support(0.008) team(0.007) skill(0.006) report(0.006)

Topic 1: experience(0.021) design(0.017) work(0.013) knowledge(0.012) development(0.011) security(0.010) team(0.009) software(0.008) product(0.007) test(0.007)

Topic 2: work(0.016) team(0.014) experience(0.014) marketing(0.009) business(0.008) skill(0.008) customer(0.007) sale(0.006) client(0.006) strong(0.005)

Topic 3: experience(0.017) datum(0.016) statistical(0.014) work(0.014) analysis(0.012) team(0.011) project(0.010) skill(0.010) study(0.008) development(0.008)

In [44]: *# Get dominant topic per document*

```
def get_dominant_topic(doc_bow):
    topics = lda.get_document_topics(doc_bow)
    return max(topics, key=lambda x: x[1])[0]

df2["dominant_topic"] = [get_dominant_topic(doc) for doc in bow_corpus]
```

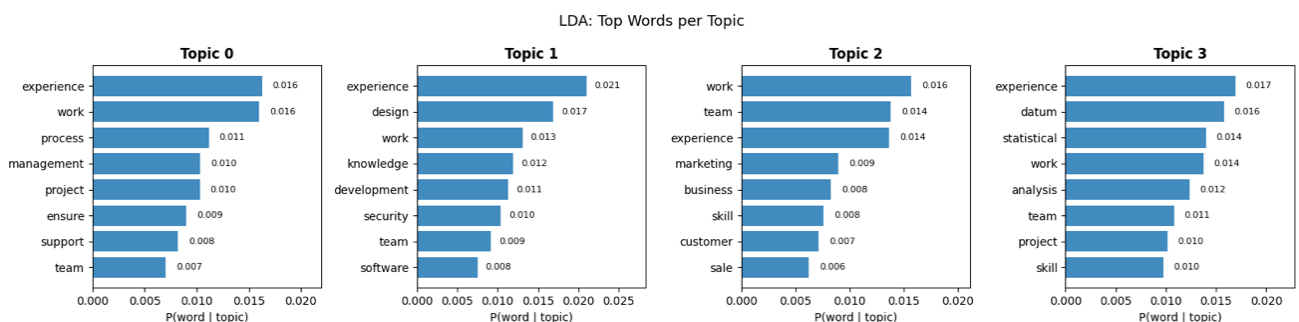
In [47]: *# Bar charts*

```
fig, axes = plt.subplots(1, K, figsize=(4*K, 4))

for t, ax in enumerate(axes):
    words, probs = zip(*lda.show_topic(t, topn=8))
    ax.barh(list(words)[:8], list(probs)[:8], alpha=0.85)
    ax.set_title(f"Topic {t}", fontsize=12, fontweight='bold')
    ax.set_xlabel("P(word | topic)")
    ax.set_xlim(0, max(probs) * 1.35)

    for i, (w, p) in enumerate(zip(reversed(words), reversed(probs))):
        ax.text(p + 0.001, i, f"{p:.3f}", va='center', fontsize=8)

plt.suptitle("LDA: Top Words per Topic", fontsize=13)
plt.tight_layout()
plt.show()
```



```
In [49]: # For each document, get its topic distribution
topic_matrix = []
for bow in bow_corpus:
    dist = dict(lda.get_document_topics(bow, minimum_probability=0))
    topic_matrix.append([dist.get(t, 0.0) for t in range(K)])
topic_matrix = np.array(topic_matrix)

topic_cols = [f"Topic {t}" for t in range(K)]
topic_prob_df = pd.DataFrame(topic_matrix, columns=topic_cols)
topic_prob_df["True major_job"] = df2["major_job"].values
topic_prob_df["Predicted topic"] = topic_prob_df[topic_cols].idxmax(axis=1)

# Reorder columns for display
topic_prob_display = topic_prob_df[["True major_job"] + topic_cols + ["Predicted topic"]]
topic_prob_display.insert(0, "Doc", range(1, len(topic_prob_display) + 1))

print("P(topic | document)")
display(topic_prob_display.round(3))
```

P(topic | document)

	Doc	True major_job	Topic 0	Topic 1	Topic 2	Topic 3	Predicted topic
0	1	SCIENCE AND ENGINEERING PROFESSIONALS	0.002	0.995	0.002	0.001	Topic 1
1	2	SCIENCE AND ENGINEERING PROFESSIONALS	0.000	0.999	0.000	0.000	Topic 1
2	3	SCIENCE AND ENGINEERING PROFESSIONALS	0.000	0.999	0.000	0.000	Topic 1
3	4	SCIENCE AND ENGINEERING PROFESSIONALS	0.009	0.006	0.008	0.977	Topic 3
4	5	SCIENCE AND ENGINEERING PROFESSIONALS	0.009	0.006	0.008	0.977	Topic 3
5	6	SCIENCE AND ENGINEERING PROFESSIONALS	0.009	0.006	0.008	0.977	Topic 3
6	7	SCIENCE AND ENGINEERING PROFESSIONALS	0.001	0.997	0.001	0.001	Topic 1
7	8	SCIENCE AND ENGINEERING PROFESSIONALS	0.001	0.997	0.001	0.001	Topic 1
8	9	SCIENCE AND ENGINEERING PROFESSIONALS	0.001	0.997	0.001	0.001	Topic 1
9	10	SCIENCE AND ENGINEERING PROFESSIONALS	0.000	0.378	0.128	0.494	Topic 3

```
In [50]: # Assign dominant topic to each document

df2["dominant_topic"] = np.argmax(topic_matrix, axis=1)

print("Dominant topic counts:")
print(df2["dominant_topic"].value_counts().sort_index())
```

```
Dominant topic counts:
dominant_topic
0      706
1      423
2      520
3      292
Name: count, dtype: int64
```

```
In [51]: # Compare dominant topic with actual major_job
```

```
topic_vs_major = pd.crosstab(
    df2["major_job"],
    df2["dominant_topic"],
    normalize="index"
).round(3)

print("==== TOPIC vs MAJOR JOB =====")
print(topic_vs_major)
```

```
==== TOPIC vs MAJOR JOB =====
dominant_topic      0      1      2
3
major_job
ADMINISTRATIVE AND COMMERCIAL MANAGERS      0.439  0.025  0.483  0.0
53
BUSINESS AND ADMINISTRATION PROFESSIONALS      0.312  0.014  0.667  0.0
07
INFORMATION AND COMMUNICATIONS TECHNOLOGY PROFE... 0.063  0.606  0.251  0.0
81
SCIENCE AND ENGINEERING PROFESSIONALS      0.505  0.158  0.069  0.2
68
```

```
In [52]: # Heatmap of topic concentration by major_job
```

```
import seaborn as sns

plt.figure(figsize=(8, 5))
sns.heatmap(topic_vs_major, annot=True, cmap="YlOrRd", vmin=0, vmax=1)
plt.title("P(dominant topic | major_job)")
plt.xlabel("Dominant Topic")
plt.ylabel("Major Job")
plt.tight_layout()
plt.show()
```

